IMPLEMENTASI DAN ANALISIS OTOMATISASI *BACKUP* DATA PADA JARINGAN *HYBRID* SISTEM PEMERIKSAAN PASIEN RAWAT INAP DI RUMAH SAKIT

SKRIPSI

Diajukan Sebagai Salah Satu Syarat Untuk Menyelesaikan
Pendidikan Jenjang Sarjana Terapan
Pada Politeknik Negeri Lhokseumawe

Oleh:

AL NAZIRUL MUBIN

NIM	**: 2022903430003**
Program Studi	**: Teknologi Rekayasa Komputer Jaringan**
Jurusan	**: Teknologi Informasi Komputer**

KEMENTERIAN PENDIDIKAN TINGGI, SAINS, DAN TEKNOLOGI
POLITEKNIK NEGERI LHOKSEUMAWE
2026

PERSETUJUAN

Skripsi yang berjudul "Implementasi dan Analisis Otomatisasi *Backup* Data pada Jaringan *Hybrid* Sistem Pemeriksaan Pasien Rawat Inap di Rumah Sakit" disusun oleh Al Nazirul Mubin, NIM 2022903430003, Program Studi Sarjana Terapan Teknologi Rekayasa Komputer Jaringan, Jurusan Teknologi Informasi dan Komptuer pada Politeknik Negeri Lhokseumawe, telah disetujui dan memenuhi syarat untuk dipertanggungjawabkan dihadapan dewan penguji.

Lhokseumawe, 20 Juli 2026

Persetujuan Pembimbing

Pembimbing Utama	Pembimbing Pendamping
Hari Toha Hidayat, S.Si., M.Cs.	**Muhammad Davi, S.Kom., M.Cs.**
NIP 198510142014041001	NIP 198905102022031006

Mengetahui

Ketua Jurusan Teknologi Informasi dan Komputer	Koordinator Program Studi Teknologi Rekayasa Komputer Jaringan
Salahuddin, S.T., M.Cs.	**Nanda Saputri, S.S.T., M.T**
NIP 197404242002121001	NIP 199111202022032010

PENGESAHAN SKRIPSI

Skripsi yang berjudul "Implementasi dan Analisis Otomatisasi *Backup* Data pada Jaringan *Hybrid* Sistem Pemeriksaan Pasien Rawat Inap di Rumah Sakit" disusun oleh Al Nazirul Mubin, NIM 2022903430003, Program Studi Sarjana Terapan Teknologi Rekayasa Komputer Jaringan, Jurusan Teknologi Informasi dan Komputer. Pada Politeknik Negeri Lhokseumawe, telah dipertanggungjawabkan didepan dewan penguji dan dinyatakan lulus pada tanggal 20 Juli 2026.

Tim Komisi Sidang

Hari Toha Hidayat, S.Si., M.Cs. NIP 198510142014041001	..... (Ketua Sidang)
Muhammad Davi, S.Kom., M.Cs. NIP 198905102022031006	..... (Sekretaris Sidang)
Amri, S.S.T., M.T. NIP 197202022000121001	..... (Penguji I)
Afla Nevrisa, S.Kom., M.Kom. NIP 199211172022032007	..... (Penguji II)
Dra. Jamilah, M.Pd. NIP 196312311993032004	..... (Penguji III)

Mengesahkan

Ketua Jurusan Teknologi Informasi dan Komputer,	Koordinator Program Studi Teknologi Rekayasa Komputer Jaringan
Salahuddin, S.T., M.Cs. NIP 197404242002121001	**Nanda Saputri, S.S.T., M.T** NIP 199111202022032010

PERNYATAAN KEASLIAN TULISAN

Saya yang bertanda tangan dibawah ini:

Nama Mahasiswa	: Al Nazirul Mubin
NIM	: 2022903430003
Program Studi	: Teknologi Rekayasa Komputer Jaringan

Menyatakan bahwa skripsi saya berjudul "Implementasi dan Analisis Otomatisasi *Backup* Data pada Jaringan *Hybrid* Sistem Pemeriksaan Pasien Rawat Inap di Rumah Sakit" ini benar-benar merupakan hasil karya saya sendiri. Selain itu, sumber informasi yang dikutip dari Penulis lain telah disebut dalam teks dan dicantumkan dalam daftar pustaka.

Apabila pada kemudian hari terbukti atau dapat dibuktikan skripsi ini hasil jiplakan, saya bersedia menerima sanksi atas perbuatan saya.

Lhokseumawe, 20 Juli 2026
Yang membuat pernyataan



Al Nazirul Mubin
NIM 2022903430003

ABSTRAK

Ketersediaan dan keutuhan data rekam medis pasien rawat inap merupakan aspek kritikal dalam operasional sistem informasi manajemen rumah sakit (SIMRS) yang sering kali terganggu oleh efek penguncian tabel (*table locking*) saat proses pencadangan konvensional. Mengatasi tantangan tersebut, penelitian ini bertujuan untuk mencegah *table locking* serta mengukur efisiensi durasi dan resiliensi sinkronisasi sistem melalui otomatisasi *backup*. Metode penelitian yang digunakan adalah kuantitatif eksperimental dengan mengimplementasikan otomatisasi pencadangan data fisik (*hot backup*) menggunakan topologi replikasi *master-slave* MariaDB yang terintegrasi dengan penjadwalan *cron job* pada ekosistem jaringan *hybrid*. Evaluasi performa sistem dilakukan menggunakan metode *average time* untuk menghitung efisiensi waktu komputasi, dan pemantauan latensi sinkronisasi menggunakan instrumen *pt-heartbeat*. Hasil penelitian membuktikan bahwa sistem berhasil mencapai target layanan tanpa jeda (*zero-downtime*) dengan rata-rata durasi eksekusi *full backup* sebesar 9,1 detik dan *incremental backup* sebesar 8,7 detik, menghasilkan tingkat efisiensi pemrosesan sebesar 4,39%. Analisis instrumen *pt-heartbeat* mengonfirmasi resiliensi latensi jaringan yang sangat stabil di angka rata-rata 54 ms pada kondisi normal, dan hanya bergeser ke 55.5 ms saat eksekusi beban tinggi, meskipun terekam lonjakan antrean sesaat hingga 385.3 ms. Walaupun pengujian *failover* sukses mengalihkan rute akses secara instan, simulasi injeksi kegagalan mengungkap kerentanan terbentuknya redundansi data usang (*stale data*) jika konflik sinkronisasi (*error* 1062) tidak termitigasi, mempertegas urgensi penerapan pemantauan sistem secara waktu nyata di masa mendatang.

Kata Kunci: *average time; backup*; *hybrid*; latensi; *master-slave*.

ABSTRACT

The availability and integrity of inpatient medical record data are critical aspects in the operation of the Hospital Management Information System (SIMRS), which are frequently disrupted by table locking effects during conventional backup processes. To overcome these challenges, this study aims to prevent table locking and measure the duration efficiency and synchronization resilience of the system through backup automation. The research method used is quantitative experimental, implementing automated physical data backup (hot backup) using a MariaDB master-slave replication topology integrated with cron job scheduling within a hybrid network ecosystem. System performance evaluation was conducted using the average time method to calculate computational time efficiency, and synchronization latency monitoring was performed using the pt-heartbeat instrument. The results proved that the system successfully achieved the zero-downtime service target with an average execution duration of 9.1 seconds for full backups and 8.7 seconds for incremental backups, resulting in a processing efficiency rate of 4.39%. Analysis of the pt-heartbeat instrument confirmed highly stable network latency resilience at an average of 54 ms under normal conditions, shifting only to 55.5 ms during high-load execution, despite recording a momentary queue spike of up to 385.3 ms. Although failover testing successfully redirected access routes instantly, fault injection simulation revealed a vulnerability regarding the formation of stale data redundancy if synchronization conflicts (error 1062) are not mitigated, thereby emphasizing the urgency of implementing real-time system monitoring in the future.

Keywords: *average time; backup; hybrid; latency; master-slave.*

KATA PENGANTAR

Segala puji beserta syukur senantiasa Penulis panjatkan kepada Allah SWT yang telah memberikan rahmat dan petunjuk-Nya kepada Penulis sehingga dapat menyelesaikan skripsi yang berjudul "Implementasi dan Analisis Otomatisasi *Backup* Data pada Jaringan *Hybrid* Sistem Pemeriksaan Pasien Rawat Inap di Rumah Sakit". Shalawat beserta salam kepada baginda Rasulullah Muhammad SAW, keluarga, dan para sahabat yang telah berjasa dalam memperjuangkan islam dan ilmu pengetahuan. Skripsi ini disusun sebagai salah satu sayarat untuk menyelesaikan pendidikan Sarjana Terapan Program Studi Teknologi Rekayasa Komputer Jaringan Jurusan Teknologi Informasi dan Komputer Politeknik Negeri Lhokseumawe.

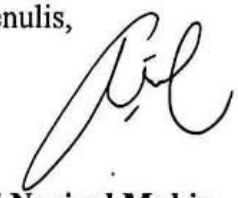
Penyusunan skripsi ini Penulis banyak mendapatkan dukungan, perhatian, dorongan, bimbingan dan berbagai bantuan dari banyak pihak terutama para dosen pembimbing. Dengan demikian, segala hormat dan kerendahan hati Penulis ini menyampaikan rasa terima kasih yang tidak terhingga terutama kepada:

1. Bapak Dr. Ir. Rizal Syahyadi, S.T., M.Eng.Sc. IPM.ASEAN Eng., APEC Eng., selaku Direktur Politeknik Negri Lhokseumawe.
2. Bapak Salahuddin, S.T., M.Cs., selaku Ketua Jurusan Teknologi Informasi dan Komputer Politeknik Negeri Lhokseumawe.
3. Bapak Fachri Yanuar Rudi F, S.S.T., M.T., selaku Sekretaris Jurusan Teknologi Informasi dan Komputer Politeknik Negeri Lhokseumawe.
4. Ibu Nanda Saputri, S.S.T., M.T., selaku Ketua Program Studi Sarjana Terapan Teknologi Rekayasa Komputer Jaringan.
5. Bapak Hari Toha Hidayat, S.Si., M.Cs. selaku Pembimbing Utama dan Bapak Muhammad Davi, S.Kom., M.Cs. selaku Pembimbing Pendamping sekaligus Koordinator Skripsi Program Studi Sarjana Terapan Teknologi Rekayasa Komputer Jaringan yang telah membimbing Penulis, memberi petunjuk, memberi saran-saran dan arahan serta masukan dari awal masa pengajuan judul sampai berhasil menyelesaikan skripsi.

6. Bapak/Ibu Dosen Penguji, yang telah berkenan menguji hasil penelitian dari Penulis, dan memberikan hal-hal terbaik bagi Penulis, kritik, saran, dan masukan agar Penulis menjadi lebih baik untuk kedepannya.
7. Bapak Anwar, S.Si., M.Cs. sebagai Pembimbing Akademik yang telah memberikan wejangan sebelum Penulis menempuh semester akhir.
8. Seluruh Dosen Pengajar Program Studi Sarjana Terapan Teknologi Rekayasa Komputer Jaringan.
9. Seluruh Staf Administrasi pada Jurusan Teknologi Informasi dan Komputer.
10. Kedua orang tua tercinta, Ayahanda Saifuddin dan Almarhumah Ibunda Salamah. Secara khusus, bait terima kasih ini Penulis dedikasikan untuk mengenang Almarhumah Ibunda tercinta, meski raga tak lagi bersama, Penulis meyakini bahwa untaian do'a, ridha, dan limpahan kasih sayang yang Ibu tanamkan semasa hidup telah menjadi lentera utama yang menuntun Penulis hingga ke titik ini. Didampingi keteguhan hati, bimbingan, serta pengorbanan materi dan moril dari Ayahanda, Penulis mampu melangkah tegar melewati setiap tantangan hingga dapat menyelesaikan skripsi ini dengan baik. Tiada untaian kata yang mampu mewakili besarnya rasa cinta, rindu, dan syukur Penulis. Semoga pencapaian ini dapat menjadi kebanggaan di alam sana serta wujud bakti kecil atas segala kebaikan yang telah diberikan.
11. Abang yang terbanggakan, Muhammad Aqhari yang memberikan dukungan kepada Penulis selama proses penyusunan skripsi ini.
12. Sahabat-sahabat seperjuangan Bayu Angkasa Putra, Muhammad Al Fatih, Syarifah Rizkyna, Fairuz Azzahira, Muhammad Rayyan, Muhammad Wahyu Rizky, Muhammad Faris Elfanda, Saiful Ahmad, Shubhan Batara Ak, Rafikul Muttakin, Irfan Kalkausar, Muhammad Tasliman, Muhammad Fadil, dan Pak Muhammad Kahfi Aulia yang saling mendukung satu sama lain dengan segala suka dan duka, terima kasih telah menjadi bagian dari perjalanan pendidikan penulis.
13. Teman-teman seperjuangan Program Studi Teknologi Rekayasa Komputer Jaringan Angkatan 2022 yang telah saling menyemangati.

Penulis menyadari masih banyak kekurangan dalam penulisan ini dan berharap skripsi ini bermanfaat bagi semua pihak serta amal baik yang diberikan kepada Penulis dibalas oleh Allah SWT, amin.

Lhokseumawe, 20 Juli 2026
Penulis,



Al Nazirul Mubin
NIM 2022903430003

DAFTAR ISI

HALAMAN JUDUL
PERSETUJUAN ..... **ii**
PENGESAHAN SKRIPSI ..... **iii**
PERNYATAAN KEASLIAN TULISAN ..... **iv**
ABSTRAK ..... **v**
ABSTRACT ..... **vi**
KATA PENGANTAR ..... **vii**
DAFTAR ISI ..... **x**
DAFTAR TABEL ..... **xiii**
DAFTAR GAMBAR ..... **xiv**
BAB I PENDAHULUAN **1**
1.1 Latar Belakang Masalah 1
1.2 Rumusan Masalah 2
1.3 Tujuan Penelitian 2
1.4 Batasan Masalah 3
1.5 Manfaat Penelitian 4
1.5.1 Manfaat bagi institusi (rumah sakit) 4
1.5.2 Manfaat akademis dan teoritis 4
1.6 Sistematika Penulisan 4
BAB II TINJAUAN PUSTAKA **6**
2.1 *State of the Art* 6
2.1.1 *Gap* yang dapat diisi 15
2.1.2 Keunggulan/*novelty* penelitian 16
2.2 Tinjauan Teoritis 17
2.2.1 Sistem dan basis data 17
2.2.2 Replikasi basis data 18
2.2.3 Arsitektur *master-slave* 19
2.2.4 MariaDB-Backup 20
2.2.5 *Cron job scheduling* 21

2.2.6 Jaringan *hybrid* di sektor kesehatan 22
2.2.7 Parameter *average time* 23
2.2.8 Parameter *Seconds_Behind_Master* pada replikasi MySQL 25
2.2.9 Tingkat efisiensi waktu backup 26
BAB III METODOLOGI PENELITIAN 28
3.1 Metode Penelitian 28
3.1.1 Tahapan penelitian 28
3.1.2 Metode pengumpulan dan analisis data 30
3.2 Data dan Pengumpulan Data 31
3.2.1 Identifikasi jenis data 32
3.2.2 Sumber data 32
3.2.3 Cara pengumpulan data 33
3.3 Rancangan Sistem (*Hardware/Software*) 33
3.3.1 Rancangan *input* 35
3.3.2 Rancangan proses 35
3.3.3 Rancangan *output* 36
3.4 Teknik Pengujian 36
3.4.1 Pengujian sinkronisasi dan konsistensi data 36
3.4.2 Pengujian otomatisasi dan *hot backup* 36
3.4.3 Pengujian ketersediaan (*failover*) 37
BAB IV HASIL DAN PEMBAHASAN 38
4.1 Hasil Implementasi Sistem 38
4.1.1 Realisasi lingkungan kerja 38
4.1.2 Hasil konfigurasi replikasi *master-slave* MySQL 39
4.1.3 Hasil implementasi otomatisasi *backup* terjadwal 41
4.2 Pengujian dan Analisis Data 44
4.2.1 Arsitektur jaringan *hybrid* dan *master-slave* 45
4.2.2 Pengujian sinkronisasi dan konsistensi data pada replikasi 47
4.2.3 Analisis latensi replikasi 52
4.2.4 Analisis efisiensi durasi otomatisasi *backup* 55
4.2.5 Pengujian penanganan kegagalan replikasi 58

4.2.6 Pengujian ketersediaan (*failover*) 59
4.3 Pembahasan 62
4.3.1 Analisis keandalan dan konsistensi replikasi *master-slave* 62
4.3.2 Latensi jaringan terhadap proses *hot hackup* 62
4.3.3 Efisiensi otomatisasi *backup* menggunakan *average time* 63
4.3.4 Implikasi kegagalan sinkronisasi replikasi 63
4.3.5 Ketersediaan data tinggi (*high availability*) dan *failover* 64
BAB V PENUTUP 65
5.1 Simpulan 65
5.2 Saran 66
DAFTAR PUSTAKA 67

DAFTAR TABEL

Tabel 2.1 Paparan *State of the Art* 7
Tabel 4.1 Spesifikasi Teknis *Virtual Private Server* (VPS)..... 38
Tabel 4.2 Parameter Kritis Status Replikasi pada *Node Slave* 40
Tabel 4.3 Konfigurasi Penjadwalan *Crontab* pada Linux *Node Slave* 43
Tabel 4.4 Komparasi Jumlah Baris dan Konsistensi Objek Basis Data 47
Tabel 4.5 Hasil Pengukuran Latensi Replikasi pada Kondisi Normal (*Idle*) 52
Tabel 4.6 Hasil Pengukuran Latensi Replikasi Saat Proses *Full Backup* 53
Tabel 4.7 Hasil Pengukuran Durasi Otomatisasi *Full Backup* 55
Tabel 4.8 Hasil Pengukuran Durasi Otomatisasi *Incremental Backup* 56
Tabel 4.9 Evaluasi Skenario Kegagalan Sinkronisasi (Konflik Data) 58
Tabel 4.10 Hasil Pengujian Skenario Ketersediaan (*Failover*) Sistem 60

DAFTAR GAMBAR

Gambar 3.1 *Flowchart* Metodologi Penelitian..... 28
Gambar 3.2 Visualisasi Arsitektur Sistem..... 35
Gambar 4.1 Penetapan server-id dan *log*_bin *Node Master*..... 39
Gambar 4.2 Penetapan server-id dan *log*_bin *Node Slave* 39
Gambar 4.3 Status Replikasi pada *Node Master*..... 39
Gambar 4.4 Baris Sintaks Konfigurasi Replikasi *Node Slave* 40
Gambar 4.5 Skrip mysql_full_backup.sh..... 41
Gambar 4.6 Skrip mysql_incremental_backup.sh 42
Gambar 4.7 Direktori Hasil *Backup* pada *Node Slave* 44
Gambar 4.8 Isi dari Folder Hasil *Backup* pada 14 Juni 2026..... 44
Gambar 4.9 Alur Sistem Jaringan *Hybrid* dan *master-slave*..... 45
Gambar 4.10 Model Kematangan Schmidt untuk Integrasi Aplikasi 46
Gambar 4.11 Kueri Validasi Baris Data pada *Node Master*..... 48
Gambar 4.12 Kueri Validasi Baris Data pada *Node Slave* 48
Gambar 4.13 Potongan Baris Data pada Tabel pasien *Node Master* 49
Gambar 4.14 Potongan Baris Data pada Tabel pasien *Node Slave* 49
Gambar 4.15 Potongan Baris Data pada Tabel users *Node Master* 50
Gambar 4.16 Potongan Baris Data pada Tabel users *Node Slave* 50
Gambar 4.17 Potongan Baris Data pada Tabel visit *Node Master*..... 51
Gambar 4.18 Potongan Baris Data pada Tabel visit *Node Slave* 51
Gambar 4.19 Grafik Komparasi Fluktuasi Latensi 54
Gambar 4.20 *Log Eror Duplicate Entry* pada Status Replikasi *Node Slave*..... 58
Gambar 4.21 Status *Slave* Ketika *Node Master* Aktif..... 61
Gambar 4.22 Status *Slave* Ketika *Node Master Down* 61

BAB I
PENDAHULUAN

1.1 Latar Belakang Masalah

Era Industri 4.0, interkoneksi sistem cerdas dan otonomi data telah menjadi pilar utama dalam modernisasi teknologi informasi. Transformasi ini mendorong adopsi jaringan *hybrid* yang menggabungkan infrastruktur lokal dan *cloud* guna menciptakan ekosistem digital yang terdistribusi dan dinamis. Dalam arsitektur tersebut, ketersediaan data menjadi aset kritikal yang harus terjamin selama 24 jam penuh untuk mendukung proses operasional yang presisi. Namun, penerapan teknologi komputasi ini membawa tantangan kompleksitas tersendiri dalam menjaga integritas informasi. Ketergantungan mutlak pada sistem digital membuat basis data sangat rentan terhadap risiko kehilangan data akibat kegagalan perangkat keras, gangguan konektivitas, maupun ancaman siber.

Integrasi teknologi informasi tersebut kini menjadi kunci efisiensi pada sektor-sektor esensial, salah satunya adalah sektor kesehatan. Rumah sakit secara progresif beralih dari pengelolaan data konvensional menuju sistem informasi manajemen rumah sakit (SIMRS) terintegrasi, khususnya pada unit rawat inap yang menghasilkan data klinis secara terus-menerus dan masif (Maulaningrum, 2025, hal: 495). Mengingat ancaman kerentanan digital yang telah disebutkan sebelumnya, institusi kesehatan menghadapi risiko yang sangat tinggi (Samant, 2024, hal: 3096). Pada layanan rawat inap, setiap detik data pasien sangat berharga, sehingga terjadinya *downtime* atau kehilangan informasi medis tidak hanya menghambat operasional administratif, tetapi juga berpotensi mengancam keselamatan pasien akibat diskontinuitas riwayat perawatan.

Implementasi strategi penggandaan data melalui metode *master-slave* pada sistem basis data menjadi solusi teknis yang sangat relevan. Metode ini memungkinkan replikasi data secara *real-time* dari server utama ke server cadangan, sehingga menciptakan penggandaan data yang andal dalam lingkungan jaringan *hybrid* (Andhavarapu, 2025, hal: 1295). Dengan skema ini, beban kerja server dapat didistribusikan secara efektif dan sinkronisasi data antar infrastruktur

tetap terjaga, yang pada akhirnya meminimalkan risiko titik kegagalan tunggal (*single point of failure*) pada basis data rumah sakit.

Penelitian ini menitikberatkan pada pengembangan sistem otomatisasi pencadangan yang terintegrasi dengan penjadwalan sistematis. Penggunaan metode pencadangan saat sistem sedang berjalan (*hot backup*) menjadi krusial agar proses pengamanan data tidak menyebabkan penguncian tabel (*table locking*) yang dapat mengganggu input data pasien (Percona LLC, 2025, hal: 8). Melalui pendekatan otomatisasi yang terjadwal, sinkronisasi data dalam jaringan *hybrid* dapat berjalan secara konsisten dan terukur, sehingga menciptakan resiliensi sistem informasi rumah sakit yang tangguh di tengah tuntutan digitalisasi kesehatan saat ini.

1.2 Rumusan Masalah

Berikut ini merupakan beberapa rumusan masalah yang diteliti:

a. Bagaimana efektivitas implementasi arsitektur replikasi *master-slave* terintegrasi *cron job* dan mariadb-*backup* dalam mengeksekusi otomatisasi *hot backup* guna mencegah efek penguncian tabel (*table locking*) pada jaringan *hybrid* sistem rawat inap rumah sakit?
b. Sejauh mana tingkat efisiensi komputasi dan resiliensi sinkronisasi sistem yang dicapai berdasarkan pengukuran durasi pencadangan menggunakan metode *average time* dan pemantauan latensi replikasi menggunakan instrumen *pt-heartbeat* pada kondisi operasional normal maupun beban tinggi?

1.3 Tujuan Penelitian

Berikut ini merupakan tujuan dari penelitian yang menjawab pertanyaan pada rumusan masalah:

a. Membuktikan efektivitas implementasi arsitektur replikasi *master-slave* terintegrasi *cron job* dan mariadb-*backup* dalam mengeksekusi otomatisasi *hot backup*, sehingga layanan input data rekam medis terhindar dari efek penguncian tabel (*table locking*) dan dapat beroperasi tanpa jeda layanan (*zero-downtime*).

b. Menghitung tingkat efisiensi komputasi dan resiliensi sinkronisasi sistem melalui pengukuran rata-rata durasi eksekusi pencadangan menggunakan metode *average time*, beserta pemantauan stabilitas latensi jaringan menggunakan instrumen *pt-heartbeat* pada kondisi operasional normal maupun saat terjadi lonjakan beban kerja.

1.4 Batasan Masalah

Berikut ini merupakan batasan masalah pada penelitian:

a. Penelitian ini berfokus pada mekanisme otomatisasi *backup* data, tidak membahas secara mendalam konfigurasi perangkat keras atau keamanan infrastruktur pada jaringan *hybrid*.
b. Sistem basis data yang digunakan dibatasi pada MySQL dengan arsitektur *master-slave.*
c. Otomatisasi *backup* dilakukan menggunakan alat mariadb-*backup* yang dikombinasikan dengan *cron job* terjadwal pada sistem operasi berbasis Linux.
d. Data yang diproses dalam penelitian ini adalah data dari sistem informasi rawat inap Rumah Sakit pada fitur pemeriksaan *visit* pasien rawat inap.
e. Pengujian ketersediaan (*availability*) hanya dilakukan pada aspek akses baca data (*read*) saat terjadi simulasi kegagalan pada server utama (*master*).
f. Penelitian ini tidak membahas teknik enkripsi dan keamanan siber data tingkat lanjut atau pemulihan data (*disaster recovery*) dalam skala geografis yang luas (*multi-region*), melainkan berfokus pada redundansi lokal dalam lingkungan jaringan rumah sakit.
g. Penggunaan metode *average time* untuk analisis sistem otomatisasi *backup* data pada server.
h. Parameter evaluasi sistem difokuskan pada nilai *average time* untuk mengukur durasi proses otomatisasi dan *Seconds_Behind_Master* untuk mengukur latensi sinkronisasi data antar server.
i. Jumlah minimal data pasien yang di-*backup* sebanyak 50 data pasien.

1.5 Manfaat Penelitian

Penelitian ini memberikan manfaat dalam pengelolaan data sistem rawat inap rumah sakit di era transformasi digital. Penerapan replikasi *master-slave* MySQL diharapkan dapat menjadi solusi efisien untuk meminimalkan risiko kehilangan data serta mempercepat proses pemulihan sistem ketika terjadi gangguan. Secara akademis, penelitian ini berkontribusi sebagai referensi pengamatan terkait implementasi *backup database* terotomatisasi pada lingkungan sistem informasi kesehatan berbasis jaringan hybrid, sekaligus memperkaya kajian tentang penerapan prinsip Industri 4.0 dalam pengelolaan data kritikal sektor kesehatan.

1.5.1 Manfaat bagi institusi (rumah sakit)

Berikut ini merupakan manfaat bagi rumah sakit:

a. Memberikan solusi perlindungan data pasien yang lebih andal melalui sistem duplikasi otomatis, sehingga meminimalkan risiko kehilangan informasi medis yang bersifat krusial.
b. Memungkinkan operasional sistem rawat inap tetap berjalan lancar tanpa gangguan (*zero downtime*) saat proses backup berlangsung, sehingga pelayanan kepada pasien tidak terhambat.
c. Mengurangi beban kerja staf IT dalam pengelolaan basis data melalui mekanisme otomatisasi yang konsisten dan terjadwal.

1.5.2 Manfaat akademis dan teoritis

Berikut ini merupakan manfaat akademis dan teoritis:

a. Menjadi rujukan bagi penelitian selanjutnya, khususnya yang berkaitan dengan implementasi replikasi database MySQL dan optimasi *hot backup* pada sistem informasi kesehatan.
b. Berkontribusi dalam pengembangan literatur mengenai penerapan teknologi Industri 4.0, khususnya dalam aspek manajemen data di lingkungan jaringan *hybrid*.

1.6 Sistematika Penulisan

Sistematika penulisan ini disusun untuk memberikan gambaran umum mengenai struktur dan alur penyusunan skripsi. Secara garis besar, naskah ini dibagi menjadi 5 (lima) bab dengan rincian singkat sebagai berikut:

BAB I PENDAHULUAN

Bab ini menguraikan latar belakang masalah, rumusan masalah, batasan masalah, tujuan penelitian, manfaat penelitian, serta sistematika penulisan skripsi.

BAB II TINJAUAN PUSTAKA

Bab ini memaparkan landasan teori dan kajian literatur yang mendasari penelitian, meliputi konsep jaringan *hybrid*, replikasi *master-slave* MariaDB, otomatisasi pencadangan, metode evaluasi *average time*.

BAB III METODOLOGI PENELITIAN

Bab ini menjelaskan tahapan dan prosedur penelitian, mulai dari perancangan arsitektur jaringan *hybrid* hingga penyusunan skenario pengujian sistem (kondisi *idle*, beban tinggi, *failover*, dan *negative testing*).

BAB IV HASIL DAN PEMBAHASAN

Bab ini menyajikan pemaparan hasil implementasi teknis, pengolahan data empiris terhadap latensi dan durasi pencadangan, uji ketersediaan data, serta analisis komprehensif mengenai penanganan kegagalan replikasi (*error handling*).

BAB V PENUTUP

Bab ini memuat simpulan akhir yang menjawab seluruh rumusan masalah penelitian secara presisi, beserta saran-saran perbaikan strategis bagi pengembangan infrastruktur basis data rumah sakit di masa mendatang.

BAB II
TINJAUAN PUSTAKA

2.1 *State of the Art*

Bagian *state of the art* ini berfungsi untuk memetakan posisi penelitian terhadap literatur ilmiah terdahulu melalui perbandingan terperinci mengenai metode, hasil, serta kontribusi teknis yang telah dicapai. Tinjauan ini mencakup analisis mendalam terhadap berbagai studi relevan, mulai dari pemanfaatan metode replikasi dalam sistem terdistribusi hingga implementasi arsitektur *master-slave* pada lingkungan virtualisasi dan *cloud computing*. Dengan mengevaluasi penelitian sebelumnya, dapat diidentifikasi keterkaitan antara pengembangan infrastruktur basis data yang andal dengan kebutuhan redundansi informasi yang semakin kompleks di era digital saat ini.

Melalui identifikasi persamaan dan perbedaan teknis tersebut, bagian ini menyoroti celah (*gap*) penelitian yang menjadi landasan utama bagi kebaruan (*novelty*) skripsi yang diusulkan. Berbeda dengan fokus penelitian terdahulu yang mayoritas menitikberatkan pada optimasi performa kueri atau mekanisme *failover* standar, penelitian ini menghadirkan solusi spesifik berupa integrasi mariadb-*backup* dan *cron job scheduling* untuk otomatisasi *hot backup* pada jaringan *hybrid*. Pendekatan ini ditujukan untuk memenuhi standar kepatuhan data kesehatan yang bersifat kritis dan sensitif, sekaligus mengatasi kompleksitas sinkronisasi dalam infrastruktur gabungan lokal dan *cloud* yang belum banyak dieksplorasi secara mendalam dalam studi-studi sebelumnya. *State of the Art* tersebut dipaparkan pada tabel 2.1 berikut.

Tabel 2.1 Paparan *State of the Art*

No	Penulis/Tahun	Judul Artikel	Metode yang Digunakan	Hasil yang Diperoleh	Perbandingan Penelitian	
					Persamaan	**Perbedaan**
1	Sholihah & Darujati, 2022	Sistem Replikasi Basis Data berdasarkan MySQL menggunakan Container Docker	Mengimplementasi-kan replikasi *master-slave* MySQL di dalam Docker Container dengan pengujian menggunakan 20 file multimedia berkapasitas 100MB melalui skenario *upload* dan *delete* data.	Replikasi otomatis berhasil dengan sempurna, data pada *Slave* tersinkronisasi sama dengan *Master*, dan perubahan data (*upload/delete*) pada *Master* langsung tereplikasi ke *Slave* secara otomatis.	Persis menggunakan MySQL dengan metode *master-slave replication* dan fokus pada otomatisasi *backup*/replikasi data dengan testing menggunakan data *real*.	Berbeda dengan penelitian tersebut yang menggunakan *Docker Container* tanpa *mariadb-backup* dan *cron job*, penelitian ini menerapkan VPS *native* dengan *mariadb-backup* dan *cron job* untuk otomatisasi *backup* pada lingkungan produksi rumah sakit.

Lanjutan Tabel 2.1

2	Muliawan & Iskandar, 2023	Penerapan Replikasi Synchronous Pada Aplikasi Data KRS Online Pada Universitas Jabal Ghafur Menggunakan PostgreSQL	Menerapkan *synchronous replication* dengan model *One-Master One-Slave* menggunakan PostgreSQL ODBMS pada aplikasi KRS *online* untuk memastikan konsistensi data secara *real-time*.	Tingkat sinkronisasi data mencapai 100% dalam pengujian, sistem berhasil meningkatkan efisiensi, *reliability*, keamanan data, dan menjamin *availability* data akademik mahasiswa.	Sama-sama menggunakan arsitektur *master-slave* pada data kritis (akademik dan kesehatan) dengan tujuan meningkatkan *availability*, *reliability*, dan implementasi sistem terdistribusi.	Penelitian tersebut fokus pada *synchronous replication* PostgreSQL di bidang pendidikan, sedangkan penelitian ini fokus pada otomatisasi *backup* MySQL berbasis *mariadb-backup* dan *cron job* di lingkungan *healthcare*.

Lanjutan Tabel 2.1

3	Kamiswara & Wijaya, 2024	Implementasi *master-slave* Sebagai Koneksi Baca Dan Tulis Database Dengan Metode Replication	Mengimplementasi-kan *master-slave replication* MariaDB pada VMware vSphere dengan Ubuntu 22.04 LTS, melakukan pemisahan operasi *READ* di *Slave* dan *WRITE* di master, dengan *load testing* menggunakan Locust dan monitoring Zabbix.	CPU *usage* menurun drastis dari 30.9% menjadi 1.7%, *memory usage* turun dari 33.9% ke 11.0%, total request meningkat 258.31%, dan response time meningkat 219.32% menunjukkan optimasi performa yang signifikan.	Identik menggunakan *master-slave replication* MySQL/MariaDB di lingkungan virtualisasi dengan fokus optimasi sistem dan monitoring performa secara berkelanjutan.	Fokus pada optimasi performa *query* dengan pemisahan operasi *READ*/*WRITE* dan *load testing* untuk meningkatkan *response time*, sedangkan penelitian ini fokus pada otomatisasi *backup* data menggunakan mariadb-*backup* dan *cron job scheduling* di VPS untuk *disaster recovery*.

Lanjutan Tabel 2.1

4	Susilo dkk., 2024	Sosialisasi Strategi Pemanfaatan Replikasi Basis Data (Master-Master dan *master-slave*) untuk Peningkatan Kinerja Aplikasi di RSIA Mutiara Bunda	Melakukan pengabdian masyarakat berupa sosialisasi dan pelatihan kepada staf IT RSIA Mutiara Bunda tentang konsep, implementasi, dan pengelolaan replikasi *database Master-slave* dan *Master-Master* untuk sistem informasi rumah sakit.	Staf IT rumah sakit berhasil memahami konsep replikasi *database*, mampu memilih model replikasi yang sesuai kebutuhan, mendapat pelatihan praktis implementasi, dan menyusun rencana implementasi replikasi di RSIA.	Persis pada konteks *healthcare*/rumah sakit dengan data medis kritis, menggunakan *master-slave replication*, dan fokus pada keandalan serta *high availability* sistem informasi rumah sakit.	Berbentuk sosialisasi dan *knowledge* transfer tentang konsep umum replikasi tanpa detail teknis implementasi, sedangkan penelitian ini merupakan implementasi teknis spesifik menggunakan mariadb-*backup* dan *cron job* untuk otomatisasi *backup* terjadwal di sistem rawat inap.

Lanjutan Tabel 2.1

5	Haryoto dkk., 2025	Evaluation and Optimization of MySQL *master-slave* Performance with Quantitative Methods on the Database of Psychology Test Applicants SIM PT XYZ at Polda Metro Jaya	Menggunakan *experimental quantitative approach* dengan MySQL *master-slave replication* di *cloud computing (pay-as-you-go), auto-scaling* dengan spesifikasi 4GB RAM hingga 12.5GB RAM, melakukan *benchmarking* untuk evaluasi performa pada data volume 4,000-5,000 *entries* per hari.	Optimasi konfigurasi server berhasil meningkatkan performa signifikan dengan *response time CREATE* 0.0245s, *GET* 0.0010s, *UPDATE* 0.0021s, *DELETE* 0.0017s, dan *system reliability* meningkat menangani 120,000 data per bulan.	Identik menggunakan MySQL *master-slave* di *cloud*/VPS *environment* dengan data volume besar, fokus pada optimasi performa, *high transaction* volume, dan *auto-scaling capability*.	Penelitian tersebut fokus pada optimasi kinerja dan *benchmarking* sistem tes psikologi tanpa *mariadb-backup* dan *cron job*, sedangkan penelitian ini fokus pada otomatisasi *backup* menggunakan *mariadb-backup* dan *cron job* untuk data medis pasien rawat inap rumah sakit.

Lanjutan Tabel 2.1

6	Hulu dkk., 2025	Optimalisasi Kinerja CMS WordPress Melalui Penerapan Replikasi Database MySQL	Mengimplementasi-kan MySQL *master-slave replication* pada WordPress CMS menggunakan *plugin* HyperDB untuk distribusi *query*, dengan monitoring NagiosXI untuk mengukur Rou*nd Trip Average* (RTA) dan status server.	Replikasi *database* menunjukkan RTAmax 5426.22ms dibanding *single database* 1774.45ms, tingkat server *down* 16.68% vs 0%, namun berhasil mengurangi risiko kehilangan data dan mengotomatisasi penggandaan *database* dengan *query handling* 29% tanpa *critical failure*.	Persis menggunakan MySQL *master-slave replication* dengan fokus pada *availability*, *reliability*, dan otomatisasi backup/replikasi untuk mencegah kehilangan data.	Penelitian tersebut menggunakan *WordPress CMS* dengan *plugin HyperDB* dan monitoring *Nagios XI* untuk optimasi performa *blogging*, sedangkan penelitian ini menggunakan *mariadb-backup* dan *cron job* untuk otomatisasi *backup* terjadwal dengan kebutuhan *compliance healthcare*.

Lanjutan Tabel 2.1

7	Intania & Rohayani, 2025	Literature Review study of Data Replication Methods in Distributed System	Melakukan *Systematic Literature Review* (SLR) terhadap 15 jurnal ilmiah untuk menganalisis berbagai metode replikasi data (*synchronous, asynchronous, multimaster, partial replication, two-way replication*, dll) dalam *distributed system* dengan pendekatan *comparative analysis*.	Penelitian tersebut mengidentifikasi *best practice* replikasi pada tiga konteks: *multi-master replication* untuk *payroll* (akurasi 80%), *synchronous replication* untuk data akademik (sinkronisasi 100%), dan *asynchronous replication* untuk keuangan properti (*error rate* di bawah 5%), serta menyediakan *guideline* pemilihan metode replikasi.	Identik membahas metode replikasi database dengan *master-slave* sebagai salah satu fokus utama dalam konteks *distributed system* dan *best practices implementation.*	Penelitian tersebut berupa *literature review* metode replikasi tanpa implementasi praktis dan tanpa membahas *mariadb-backup* atau *cron job*, sedangkan penelitian ini implementasi praktis *master-slave* menggunakan *mariadb-backup* dan *cron job* untuk otomatisasi *backup* di *healthcare*.

Lanjutan Tabel 2.1

8	Hidayat dkk., 2026	Design and Analysis of a Mobile Chatbot Application for Hospital Room Availability Using the Bryant Evaluation Method	Mengembangkan *chatbot WhatsApp* terintegrasi SIMRS menggunakan *Bryant Method* untuk prioritasi keputusan, Node.js sebagai *engine chatbot*, *RESTful API* dan JSON sebagai *middleware*, serta basis data SQL terpusat dengan tiga model integrasi (*API-based*, *IoT-based bed sensor*, manual) pada RSIA Abby dan RSU MMC.	*Response time* rata-rata 1 detik per 750 proses; 41,3%/37% setuju ketersediaan ruang tampil *real-time*; 45,7%/30,4% setuju kecepatan respons baik; 43,5%/37% setuju chatbot membantu pemilihan ruang efisien.	Identik pada konteks rumah sakit dengan data rawat inap pasien (SANGAT RELEVAN), menggunakan integrasi sistem informasi rumah sakit (SIMRS), fokus pada *real-time* data *availability*, implementasi *database* terpusat dengan sinkronisasi, serta *multiple hospital integration* dengan arsitektur terdistribusi.	Penelitian tersebut fokus pada *chatbot interface* ketersediaan ruang dengan *Bryant Method* tanpa otomatisasi *backup*, sedangkan penelitian ini fokus pada otomatisasi *backup* pasien rawat inap menggunakan *MySQL master-slave replication*, *mariadb-backup*, dan *cron job* untuk *disaster recovery*.

2.1.1 *Gap* yang dapat diisi

Berikut ini *gap* (celah) pada masing-masing penelitian terdahulu yang sudah dirangkum pada tabel 2.1 sebelumnya:

a. Celah pada Skalabilitas Lingkungan Produksi

   Penelitian Sholihah & Darujati (2022) berfokus pada isolasi lingkungan menggunakan *Docker Container*, namun belum menyentuh penggunaan VPS *native* dengan penjadwalan *incremental backup* otomatis yang krusial untuk skalabilitas database rumah sakit pada lingkungan produksi.

b. Celah pada Pemulihan Data Spesifik (*Point-in-Time Recovery*)

   Penelitian Muliawan & Iskandar (2023) memprioritaskan konsistensi data akademik melalui replikasi *synchronous*, sehingga terdapat kekosongan dalam pembahasan mengenai mekanisme *point-in-time recovery* dan *hot backup* otomatis yang sangat dibutuhkan untuk menjaga ketersediaan data pasien yang bersifat kritis.

c. Celah Fokus Perlindungan Data (*Data Protection*)

   Meskipun penelitian Kamiswara & Wijaya (2024) berhasil mengoptimalkan performa kueri melalui pemisahan operasi baca-tulis, terdapat celah dalam penyediaan mekanisme perlindungan data otomatis untuk skenario pemulihan bencana (*disaster recovery*) pada infrastruktur VPS *cloud*.

d. Celah Kedalaman Teknis di Sektor Kesehatan

   Penelitian Susilo dkk. (2024) yang dilakukan di lingkungan rumah sakit masih bersifat sosialisasi dan pelatihan umum, menciptakan celah besar dalam hal rincian teknis implementasi otomatisasi sistem pencadangan terintegrasi untuk data rawat inap.

e. Celah Prosedur Backup Tanpa Penguncian (*Non-Locking*)

   Studi Haryoto dkk. (2025) yang berfokus pada *benchmarking* performa sistem tes tidak membahas prosedur *backup* otomatis yang menjamin operasional tetap berjalan tanpa risiko penguncian tabel (*table locking*) di tengah beban transaksi yang tinggi.

f. Celah Kepatuhan Database Murni

Penelitian Hulu dkk. (2025) menggunakan *plugin* pada CMS WordPress, sehingga belum mampu menjawab kebutuhan sistem informasi kesehatan murni yang memerlukan kontrol tingkat tinggi melalui otomatisasi di level server untuk memenuhi standar kepatuhan data medis.

g. Celah Implementasi Praktis dan Alat Otomatisasi

Sebagian besar studi literatur, seperti penelitian Intania & Rohayani (2025), masih terbatas pada tinjauan teoritis mengenai metode replikasi tanpa menyajikan implementasi praktis alat spesifik seperti mariadb-*backup* dan *cron job* dalam lingkungan medis yang nyata.

h. Celah Redundansi Sisi Belakang (*Backend Redundancy*)

Penelitian Hidayat dkk. (2026) sangat relevan dalam konteks rumah sakit namun berfokus pada antarmuka pengguna (*chatbot*) untuk informasi kamar, sehingga terdapat celah dalam strategi redundansi data di sisi belakang (*backend*) dan pengamanan berkas rekam medis pasien rawat inap.

2.1.2 Keunggulan/*novelty* penelitian

Novelty disusun untuk menunjukkan adanya hal baru, unik, atau inovatif yang ditawarkan dari penelitian sebelumnya, dipaparkan sebagai berikut:

a. Penggunaan mariadb-*backup* + *cron job* untuk otomatisasi *backup* pada MySQL *master-slave* di lingkungan VPS belum banyak dibahas dalam literatur.

b. Data sistem pemeriksaan pasien rawat inap RS memiliki karakteristik unik (HIPAA *compliance*, data sensitif, *real-time critical*) yang berbeda dari kasus LPSE, akademik, atau *e-commerce*.

c. Integrasi dengan jaringan hybrid sistem pemeriksaan pasien menambah kompleksitas dan relevansi praktis.

d. *Cron job* terjadwal untuk *backup* terjadwal berbeda dari fokus *failover*/*performance* yang dominan di penelitian terdahulu.

e. Implementasi di VPS *production-grade* vs virtualisasi lokal (VirtualBox/VMware).

2.2 Tinjauan Teoritis

Tinjauan teoritis ini menyajikan landasan konseptual dan kerangka ilmiah yang menjadi dasar dalam pengembangan sistem otomatisasi pencadangan data pada jaringan *hybrid* rumah sakit. Pembahasan mencakup prinsip dasar *Database Management System* (DBMS) sebagai perangkat lunak pengelola data serta teknik replikasi untuk menjamin konsistensi dan ketersediaan data melalui penggandaan objek basis data antar server. Selain itu, dipaparkan pula mekanisme teknis arsitektur *master-slave* pada MySQL serta peran teknologi pendukung seperti mariadb-*backup* dan *cron job* untuk mencapai efisiensi operasional tanpa mengganggu layanan medis. Pemahaman terhadap teori-teori ini sangat penting guna mewujudkan infrastruktur data yang tangguh dan selaras dengan tuntutan digitalisasi kesehatan di era Industri 4.0.

2.2.1 Sistem dan basis data

Sistem merupakan sekumpulan komponen yang saling berinteraksi, terkait, dan bekerja sama secara terorganisir untuk mencapai tujuan tertentu. Dalam perspektif teknologi informasi, sebuah sistem beroperasi melalui proses transformasi data masukan (*input*) menjadi informasi keluaran (*output*) yang bernilai. Keberadaan sebuah sistem yang terstruktur dengan baik sangat krusial, terutama pada lingkungan dengan alur kerja yang masif dan kompleks, di mana setiap komponen perangkat keras, perangkat lunak, dan pengguna harus berjalan secara harmonis dan terstandarisasi.

Sebagai fondasi utama dari sistem informasi tersebut, basis data hadir untuk menangani kompleksitas penyimpanan informasi secara digital. Basis data menjadi komponen dasar yang mengonsolidasikan catatan-catatan logis yang saling berhubungan pada suatu wadah penyimpanan terpusat. Pendekatan sentralisasi ini memungkinkan sekumpulan data yang sebelumnya tersebar menjadi terstruktur, sehingga dapat diakses, dikelola, dan diperbarui oleh beberapa pengguna secara bersamaan dengan meminimalisasi risiko redudansi.

Untuk menjembatani interaksi antara pengguna, aplikasi, dan basis data itu sendiri, diperlukan sebuah lapisan perangkat lunak khusus. Pengelolaan basis data dilakukan melalui *Database Management System* (DBMS), yaitu perangkat lunak

yang memungkinkan pengguna untuk membuat, memelihara, dan mengontrol akses data secara efisien (Kag, 2023, hal: 42). Lapisan sistem ini bertugas memastikan integritas, keamanan, dan ketersediaan data tetap terjaga, serta mengeksekusi instruksi kueri secara presisi meskipun basis data diakses oleh banyak entitas secara konkuren.

Dalam konteks operasional Sistem Informasi Manajemen Rumah Sakit (SIMRS), pemahaman terhadap arsitektur sistem dan basis data menjadi landasan mutlak. Basis data pemeriksaan pasien rawat inap menyimpan entitas medis yang bersifat kritikal, sensitif, dan menuntut ketersediaan tinggi (*high availability*). Oleh karena itu, arsitektur dasar basis data ini harus dirancang sedemikian rupa agar siap diintegrasikan dengan skema replikasi terdistribusi dan mekanisme pencadangan otomatis, guna mencegah kehilangan data di dalam jaringan *hybrid*.

2.2.2 Replikasi basis data

Replikasi adalah teknik penyalinan dan pendistribusian data serta objek basis data dari satu *database* utama ke *database* sekunder. Mekanisme ini tidak sekadar melakukan penyalinan data mentah secara buta, melainkan memastikan adanya proses sinkronisasi lanjutan agar konsistensi data tetap terjamin di seluruh titik penyimpanan. Tujuan mendasar dari replikasi adalah menciptakan redundansi data yang terstruktur, sehingga informasi yang sama dapat diakses secara simultan melalui simpul (*node*) yang berbeda tanpa merusak integritas pusat data aslinya.

Dalam lingkungan komputasi terdistribusi, berbagai strategi replikasi telah dipelajari dan diimplementasikan untuk menyimpan salinan data di beberapa lokasi secara strategis. Pendekatan ini memungkinkan permintaan data dipenuhi dari salinan server terdekat untuk memangkas latensi (waktu tunda) jaringan dan meningkatkan metrik ketersediaan (*availability*). Di saat yang bersamaan, arsitektur ini dituntut untuk terus menjaga konsistensi antar replika melalui mekanisme propagasi yang teratur dan andal (D & A, 2024, hal: 83).

Implementasi teknik replikasi menjadi sangat krusial dalam sistem terdistribusi modern sebagai lapis pertahanan utama terhadap risiko *single point of failure* (titik kegagalan tunggal). Ketika sebuah basis data diakses secara masif, pemisahan beban kerja melalui server replika secara signifikan mampu

meningkatkan kinerja dan waktu respons aplikasi. Terlebih lagi, replikasi berfungsi sebagai mekanisme pemulihan bencana (*disaster recovery*) yang menjamin kelangsungan operasional sistem, terutama saat server lokal mengalami kegagalan perangkat keras, malfungsi sistem, maupun gangguan konektivitas jaringan.

Konsep tingkat ketersediaan tinggi (*high availability*) yang ditawarkan oleh replikasi ini berbanding lurus dengan kebutuhan esensial institusi kesehatan seperti rumah sakit. Ketersediaan aplikasi maupun data yang sifatnya berkelanjutan merupakan hal mutlak dalam ekosistem sistem pemeriksaan pasien rawat inap yang tidak mentoleransi adanya waktu henti (*downtime*). Oleh karena itu, diterapkan teknik replikasi ini pada jaringan *hybrid* agar aliran pertukaran data medis dapat terus berjalan tanpa hambatan, sekaligus mempersiapkan landasan yang terisolasi bagi rutinitas pencadangan (otomasisasi *backup*).

2.2.3 Arsitektur *master-slave*

Arsitektur *master-slave* merupakan metode penggandaan basis data asimetris yang menugaskan satu server sebagai *Master* (induk) dan server lainnya sebagai *Slave* (salinan). Konsep utama dari arsitektur ini adalah membangun hierarki distribusi data satu arah (unidireksional), di mana otoritas utama (sumber kebenaran tunggal) berada pada titik pusat. Melalui topologi ini, sistem tidak hanya sekadar menduplikasi informasi, tetapi juga menciptakan struktur pendelegasian tugas yang terisolasi dan terspesialisasi antar server dalam sebuah jaringan.

Dalam model operasionalnya, pembagian peran diatur secara ketat untuk mengoptimalkan kinerja infrastruktur basis data. Server *Master* bertindak sebagai titik tumpu utama yang aktif secara eksklusif melayani seluruh antrean transaksi kritis, khususnya kueri tulis (*write*) dan manipulasi data secara langsung. Sementara itu, server *Slave* memegang peran pelengkap sebagai server cadangan siaga siaga yang bertugas proaktif mengikuti riwayat perintah *Master*, serta dapat dimanfaatkan untuk menangani distribusi beban kueri baca (*read-only*) guna mencegah kondisi kelebihan muatan (*overload*) pada peladen utama.

Keandalan arsitektur ini sangat bergantung pada presisi mekanisme sinkronisasi data yang berjalan di latar belakang jaringan. Penelitian yang mengevaluasi replikasi MySQL *master-slave* menunjukkan bahwa model ini

memastikan data yang dimasukkan atau diubah di sisi *Master* secara otomatis diteruskan dan diterapkan ke sisi *Slave*, sehingga salinan data pada *Slave* selalu konsisten dengan *Master* dalam skenario sistem terdistribusi (Haryoto, 2025, hal: 8900). Jika terjadi pembaruan, penyisipan, atau penghapusan catatan medis pada *Master*, aliran *binary log* akan seketika memaksa data pada *Slave* untuk ikut berubah, menjaga integritas informasi antar lokasi tetap terjaga secara presisi.

Relevansi penerapan topologi asimetris ini menjadi sangat strategis dalam ekosistem sistem rawat inap rumah sakit yang menuntut resiliensi tinggi. Dengan memposisikan pangkalan data transaksi operasional pada *Master* dan menjadikan *Slave* sebagai bayangannya, sistem mampu memastikan kelancaran layanan medis berkesinambungan. Pendekatan arsitektural ini yang menjadi landasan utama bagi penelitian ini untuk menyediakan lingkungan komputasi terisolasi di server cadangan, di mana proses otomatisasi pencadangan dapat dieksekusi dengan aman tanpa memicu anomali penguncian tabel (*table locking*) yang dapat menghentikan lalu lintas layanan pasien.

2.2.4 MariaDB-Backup

Mariadb-*backup* (sebelumnya dikenal dengan nomenklatur mariabackup) merupakan perangkat lunak *open-source* yang dirancang secara spesifik untuk mengeksekusi proses pencadangan data (*backup*) fisik pada ekosistem basis data MariaDB maupun MySQL. Secara historis, utilitas ini merupakan turunan (*fork*) langsung dari perangkat lunak Percona XtraBackup, yang kemudian dikembangkan dan disempurnakan oleh komunitas MariaDB guna memberikan tingkat dukungan, integrasi, dan keamanan yang lebih optimal.

Keunggulan komparatif yang paling menonjol dari mariadb-*backup* dibandingkan metode ekspor logis konvensional (seperti *mysqldump*) adalah kemampuannya dalam mengeksekusi prosedur *hot backup*. Pendekatan *hot backup* ini memungkinkan mesin untuk menyalin berkas-berkas data fisik (*physical backup*) langsung dari tingkat sistem operasi tanpa mengharuskan layanan basis data dimatikan. Lebih jauh lagi, proses penyalinan ini bekerja melalui mekanisme *non-locking*, yang berarti tabel-tabel basis data tidak mengalami penguncian secara total selama siklus pencadangan berlangsung.

Selain unggul dalam pencadangan penuh (*full backup*), mariadb-*backup* juga dibekali dengan kapabilitas *incremental backup* yang sangat adaptif. Fitur lanjutan ini beroperasi dengan memindai algoritma *log* dan hanya menyalin entri data atau blok perubahan yang terjadi sejak siklus pencadangan valid terakhir kali dieksekusi. Melalui metode inkremental ini, sistem terhindar dari keharusan menduplikasi keseluruhan volume basis data secara berulang kali, sehingga penggunaan ruang penyimpanan menjadi jauh lebih efisien dan beban kerja prosesor peladen dapat ditekan seminimal mungkin (Percona LLC, 2025, hal: 8).

Dalam konteks operasional rumah sakit, perpaduan fitur *hot backup* dan algoritma inkremental ini merupakan spesifikasi teknis yang krusial. Karakteristik non-interuptif dari mariadb-*backup* memastikan bahwa petugas medis tetap dapat melakukan *input* data diagnostik dan catatan rawat inap pasien secara *real-time* ke server *Master*, meskipun server *Slave* sedang sibuk melakukan rutinitas pengamanan data di latar belakang. Melalui implementasi alat ini, arsitektur yang dibangun mampu memberikan jaminan redundansi data medis tanpa harus mengorbankan kinerja, kecepatan, maupun kesinambungan layanan kesehatan administratif.

2.2.5 *Cron job scheduling*

Cron job adalah utilitas berbasis waktu yang tertanam secara bawaan pada sistem operasi Linux, yang berfungsi untuk menjalankan suatu tugas, perintah, atau skrip secara otomatis pada interval waktu tertentu. Utilitas ini beroperasi di latar belakang sistem dengan memantau jadwal yang telah didefinisikan, sehingga memungkinkan administrator menjalankan proses sistem secara berkala tanpa pengawasan langsung. Kehadiran mekanisme penjadwalan ini menjadi fondasi utama dalam menciptakan sistem komputasi yang otonom, efisien, dan tidak bergantung pada eksekusi manual.

Dalam perancangan sistem ini, *cron job* berperan vital sebagai penggerak utama (*automation tool*) yang mengoordinasikan seluruh alur kerja perlindungan data. Utilitas ini bertugas mengeksekusi skrip *bash* yang memicu perintah mariadb-*backup* secara mandiri, sehingga rutinitas pencadangan fisik dapat berjalan tepat waktu tanpa memerlukan intervensi manual dari staf IT rumah sakit. Integrasi

antara utilitas penjadwal dan perangkat lunak pencadangan ini membentuk sebuah siklus pengamanan data yang berjalan secara kontinu dan terukur.

Keandalan otomatisasi berbasis waktu ini telah banyak dibuktikan dalam berbagai literatur teknologi informasi modern. Penelitian terdahulu menunjukkan bahwa penggunaan *cron job* sebagai penjadwal tugas otomatis dapat secara signifikan mengurangi beban kerja manual dan meningkatkan konsistensi, reliabilitas, serta integritas proses data dalam sistem berbasis *cloud* maupun terdistribusi, melalui serangkaian skrip yang dieksekusi secara berkala (Wibowo, 2025, hal: 11). Sinkronisasi operasional ini diatur secara sistematis melalui konfigurasi *crontab* pada Linux untuk memastikan proses pencadangan data medis pada jaringan *hybrid* berjalan tanpa celah.

Melalui penerapan otomatisasi ini, potensi risiko kelalaian manusia (*human error*) dalam manajemen tata kelola peladen dapat ditekan seminimal mungkin. Kelalaian seperti keterlambatan mengeksekusi *backup* manual atau kesalahan parameter perintah dapat dieliminasi secara mutlak oleh presisi waktu sistem. Dengan demikian, keberlangsungan, ketersediaan, dan keamanan jangka panjang dari rekam medis pasien rawat inap dapat terus terjaga secara konsisten sesuai dengan kebijakan tata kelola data yang telah ditetapkan institusi kesehatan.

2.2.6 Jaringan *hybrid* di sektor kesehatan

Transformasi digital di sektor kesehatan saat ini telah mendorong pergeseran paradigma dari penggunaan infrastruktur teknologi informasi konvensional menuju adopsi jaringan *hybrid*. Ekosistem jaringan ini secara strategis menggabungkan infrastruktur lokal (*on-premises*) yang berada di lingkungan rumah sakit dengan fleksibilitas pusat data komputasi awan (*cloud*). Integrasi kedua platform ini dirancang untuk menciptakan efisiensi operasional layanan medis, di mana pengelolaan informasi tidak lagi dibatasi oleh kapasitas fisik perangkat keras tunggal, melainkan didistribusikan secara dinamis sesuai beban kerja sistem.

Implementasi *hybrid cloud architecture* ini tidak terlepas dari kebutuhan institusi medis untuk mengamankan data sensitif tanpa mengorbankan kecepatan akses. Arsitektur yang memadukan sumber daya *on-premises* dan *cloud* publik ini telah diteliti secara mendalam sebagai pendekatan strategis untuk modernisasi

sistem layanan kesehatan. Pendekatan hibrida ini dinilai sangat krusial karena mampu menyeimbangkan kebutuhan skalabilitas penyimpanan, standar keamanan data tingkat tinggi, dan kinerja operasional yang optimal dalam tata kelola data medis, serta mendukung integrasi aplikasi-aplikasi yang bersifat kritikal (Devadas, 2025, hal: 629).

Dalam realitas operasionalnya, topologi jaringan *hybrid* menjadi tulang punggung bagi kelancaran fungsi Sistem Informasi Manajemen Rumah Sakit (SIMRS). Fasilitas rawat inap memproduksi data klinis, riwayat diagnosis, dan rekam medis pasien secara masif dan terus-menerus setiap waktu. Dengan mendistribusikan lalu lintas data di antara jaringan lokal dan awan, sistem secara proaktif meminimalkan risiko kegagalan layanan atau waktu henti (*downtime*) yang sangat rentan terjadi apabila operasional rumah sakit hanya bergantung mutlak pada satu infrastruktur tunggal.

Untuk mengoptimalkan keandalan jaringan *hybrid* tersebut, diperlukan sebuah skema redundansi basis data yang terstruktur secara ketat. Implementasi strategi *master-slave* pada lingkungan *hybrid* bertujuan secara spesifik untuk memitigasi risiko titik kegagalan tunggal (*single point of failure*). Melalui perpaduan teknologi komputasi awan dan replikasi ini, ketersediaan data pemeriksaan pasien dapat dipertahankan selama 24 jam penuh untuk mendukung proses pengambilan keputusan klinis oleh tenaga medis secara akurat dan responsif.

2.2.7 Parameter *average time*

Parameter *average time* atau *mean execution time* merupakan salah satu instrumen pengukuran statistik dasar yang esensial dalam ranah pengujian infrastruktur teknologi informasi. Parameter ini secara umum digunakan untuk menilai dan mengkuantifikasi kinerja suatu sistem, aplikasi, maupun proses komputasi berdasarkan serangkaian pencatatan waktu yang berulang. Keberadaan parameter pengukuran ini memberikan landasan matematis yang objektif bagi para peneliti untuk mengevaluasi seberapa cepat sebuah instruksi dapat diselesaikan oleh peladen, sehingga simpulan performa sistem tidak dibangun di atas asumsi yang kualitatif.

Secara prinsip dasar statistika, *average* atau *mean* didefinisikan sebagai hasil bagi antara jumlah keseluruhan nilai dalam suatu kumpulan data observasi dengan total banyaknya sampel pengujian tersebut. Rata-rata waktu dihitung menggunakan formula matematis berikut:

$$Average\ Time = \frac{\sum_{i=1}^{n} Durasi\ Backup_i}{n} \quad (2.1)$$

Berikut adalah penjelasan mengenai komponen rumus tersebut:

a. *Average Time* = Hasil perhitungan yang menunjukkan nilai rata-rata waktu yang dibutuhkan sistem untuk menyelesaikan satu siklus pencadangan otomatis.
b. ∑ (Sigma) = Simbol penjumlahan untuk seluruh data durasi yang diperoleh selama masa pengujian.
c. $Durasi\ Backup_i$ = Nilai waktu (biasanya dalam satuan detik atau menit) yang dicatat pada setiap sesi *backup* ke-*i* berdasarkan catatan *log* sistem.
d. n = Total frekuensi atau jumlah pengujian *backup* yang dilakukan selama periode observasi.

Formula kalkulasi matematis ini dirancang secara spesifik untuk mengekstraksi sebuah nilai tunggal yang secara valid mampu mewakili kecenderungan pusat (*central tendency*) dari seluruh durasi yang direkam. Melalui pemusatan titik data ini, dapat diperoleh angka representatif yang paling mendekati probabilitas umum dari perilaku sistem saat beroperasi di lingkungan produksi nyata.

Keunggulan utama dari penggunaan nilai *average time* dalam evaluasi performa sistem terletak pada kemampuannya untuk menetralisasi bias data yang disebabkan oleh anomali komputasi. Dengan menggunakan nilai rata-rata, fluktuasi ekstrem pada durasi pengukuran tunggal seperti lonjakan beban CPU sesaat atau *delay* jaringan yang tidak terduga dapat dihaluskan secara proporsional. Karakteristik peredaman anomali inilah yang membuat estimasi waktu yang diperoleh menjadi lebih stabil, sehingga komparasi kinerja antar konfigurasi sistem dapat dipertanggungjawabkan secara objektif dan ilmiah (Florensia dkk., 2024, hal: 31).

Dalam konteks penelitian sistem informasi dan infrastruktur basis data, *average time* difungsikan untuk membedah tingkat efisiensi mekanisme operasional beserta dampaknya terhadap keandalan layanan secara keseluruhan. Metrik evaluasi ini sangat ideal untuk memonitor variabel-variabel krusial seperti durasi eksekusi *backup* otomatis, perhitungan latensi sinkronisasi replikasi, hingga waktu respons (*response time*) pada layanan aplikasi klinis. Oleh karena itu, pendekatan rata-rata waktu ini diadopsi sebagai pisau analisis utama guna menguji tingkat efisiensi serta ketangguhan arsitektur pencadangan yang berjalan pada jaringan *hybrid* rumah sakit.

2.2.8 Parameter *Seconds_Behind_Master* pada replikasi MySQL

Dalam arsitektur replikasi basis data MySQL, pemantauan kinerja sinkronisasi antarpeladen membutuhkan instrumen pengukuran yang presisi dan dapat dipantau secara waktu nyata (*real-time*). Untuk memenuhi kebutuhan tersebut, parameter *Seconds_Behind_Master* hadir sebagai salah satu metrik kunci yang memegang peranan krusial. Metrik bawaan dari sistem informasi MySQL ini difungsikan secara spesifik untuk memantau, mendeteksi, dan mengukur tingkat latensi atau waktu tunda replikasi yang terjadi pada jalur komunikasi antara server *Master* dan server *Slave*.

Secara teknis, kalkulasi nilai pada parameter ini didasarkan pada perbandingan stempel waktu (*timestamp*) dari setiap instruksi data atau kejadian (*event*) yang mengalir di dalam sistem. Nilai yang ditampilkan menunjukkan estimasi jumlah detik keterlambatan eksekusi peristiwa pada antrean server *Slave* apabila dikomparasikan dengan waktu pencatatan asli transaksi tersebut di server *Master*. Melalui pembacaan *timestamp* dari *binary log* dan *relay log* yang sedang diproses oleh *SQL thread*, parameter ini secara akurat mencerminkan seberapa jauh peladen salinan tertinggal dalam menerapkan perubahan data medis.

Interpretasi dari besaran nilai yang dihasilkan oleh metrik ini menjadi indikator langsung terhadap tingkat kesehatan sistem replikasi. Ketika nilai *Seconds_Behind_Master* menunjukkan angka 0 (nol), server *Slave* dianggap telah berhasil mengejar seluruh antrean kueri dan berstatus selaras seutuhnya dengan server *Master*. Sebaliknya, kemunculan nilai positif mengindikasikan adanya

keterlambatan pemrosesan yang umumnya dipicu oleh gangguan konektivitas jaringan, beban kerja memori yang terlalu tinggi, atau hambatan operasi baca-tulis perangkat keras (*disk I/O*) pada proses replikasi (Percona LLC, 2025, hal: 8).

Pemahaman dan pemantauan terhadap parameter ini memegang peranan yang sangat penting dalam pengujian arsitektur sistem basis data terdistribusi. Nilai fluktuasi latensi yang terekam dapat diekstraksi untuk mengevaluasi tingkat presisi sinkronisasi data serta mengukur batas keandalan sistem replikasi secara kuantitatif. Dalam konteks pengelolaan data rawat inap rumah sakit, keberadaan metrik ini sangat esensial untuk dimanfaatkan di dalam penelitian ini guna mengukur sejauh mana dampak dari beban kerja tinggi terutama saat proses pencadangan berlangsung terhadap ketersediaan informasi medis yang mutakhir pada server cadangan.

2.2.9 Tingkat efisiensi waktu backup

Efisiensi waktu merupakan salah satu indikator kinerja yang lazim digunakan dalam evaluasi sistem komputer untuk mengukur seberapa besar penghematan waktu yang diperoleh dari penerapan suatu metode dibandingkan dengan metode lain yang dijadikan acuan (*baseline*). Pengukuran ini dilakukan dengan membandingkan durasi eksekusi antara metode yang dievaluasi dengan durasi eksekusi metode acuan, kemudian menyatakan selisihnya dalam bentuk persentase. Semakin besar persentase yang dihasilkan, semakin besar pula penghematan waktu yang dicapai oleh metode yang dievaluasi terhadap metode acuannya.

Dalam konteks sistem pencadangan data, konsep tersebut diterapkan untuk membandingkan durasi proses antara metode yang menyalin keseluruhan data (*full backup*) dengan metode yang hanya menyalin data yang mengalami perubahan sejak pencadangan terakhir (*incremental backup*). Oleh karena *incremental backup* memproses volume data yang jauh lebih kecil, durasi eksekusinya secara umum lebih singkat dibandingkan *full backup*, sehingga selisih waktu di antara keduanya dapat dinyatakan sebagai tingkat efisiensi waktu (*time efficiency rate*) dari penerapan metode *incremental backup*.

Secara matematis, tingkat efisiensi waktu dirumuskan sebagai persentase selisih waktu proses terhadap waktu dasar (*baseline*), sebagaimana rumus yang diadaptasi dari Dhafa dan Kiswanto (2026, hal: 1086) berikut.

$$Tingkat\ Efisiensi = \frac{T_{full} - T_{incremental}}{T_{full}} \text{ X } 100\% \qquad (2.2)$$

Keterangan:

a. T_{full} = Rata-rata waktu eksekusi *full backup* (detik)

b. $T_{incremental}$ = Rata-rata waktu eksekusi *incremental backup* (detik)

Nilai persentase yang dihasilkan dari perhitungan tersebut menggambarkan besarnya penghematan waktu yang diperoleh dari penerapan metode *incremental backup* dibandingkan *full backup*; semakin tinggi nilai persentasenya, semakin efisien metode pencadangan yang diterapkan dalam hal kecepatan eksekusi. Pendekatan pengukuran ini bersifat umum dan dapat diterapkan pada berbagai lingkungan sistem pencadangan data, termasuk pada sistem basis data yang menerapkan mekanisme replikasi maupun *hot backup*, selama kedua metode yang dibandingkan diuji pada kondisi lingkungan dan volume data yang setara agar hasil pengukuran tetap valid dan dapat dipertanggungjawabkan.

Pada penelitian ini, rumus tingkat efisiensi waktu tersebut digunakan untuk mengukur perbandingan durasi eksekusi antara *full backup* dan *incremental backup* yang dijalankan melalui mariadb-*backup* pada basis data rawat_inap_vps. Waktu eksekusi *full backup* yang dijalankan setiap pukul 02.00 dijadikan sebagai *baseline*, sedangkan waktu eksekusi *incremental backup* yang dijalankan pada pukul 09.00, 15.00, dan 21.00 dijadikan sebagai waktu yang dibandingkan. Hasil perhitungan tingkat efisiensi waktu tersebut selanjutnya digunakan untuk menganalisis seberapa besar kontribusi penerapan *incremental backup* terhadap efisiensi waktu pemrosesan pencadangan data pada sistem informasi manajemen rawat inap yang berjalan di lingkungan VPS.

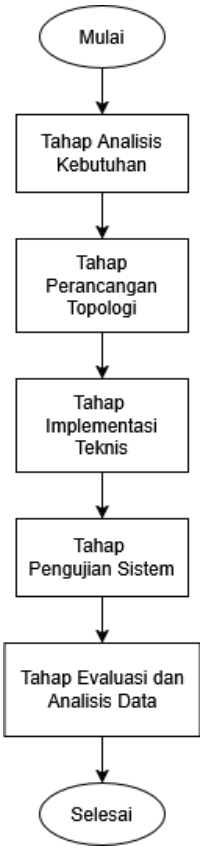
BAB III
METODOLOGI PENELITIAN

3.1 Metode Penelitian

Metode penelitian ini disusun secara sistematis untuk menguji keandalan sistem otomatisasi *backup* pada jaringan *hybrid* sistem pemeriksaan pasien rawat inap. Pendekatan yang digunakan adalah penelitian kuantitatif eksperimental yang mencakup tahapan analisis kebutuhan, perancangan topologi, hingga implementasi teknik replikasi *master-slave* pada lingkungan *Virtual Private Server* (VPS). Melalui penetapan variabel yang terukur, penelitian ini bertujuan untuk menganalisis efisiensi durasi *backup* serta stabilitas sinkronisasi data medis secara otomatis guna menjamin konsistensi dan ketersediaan informasi yang berkelanjutan.

3.1.1 Tahapan penelitian

Alur metodologi penelitian dapat dilihat pada *flowchart* gambar 3.1 berikut,



Gambar 3.1 *Flowchart* Metodologi Penelitian

Dilihat pada gambar 3.1 yang menunjukkan metodologi yang digunakan adalah adaptasi dari model pengembangan sistem yang sistematis:

a. Tahap analisis kebutuhan

Peneliti melakukan identifikasi data yang diterima dari jaringan *hybrid* sistem pemeriksaan pasien rawat inap. Rasionalnya adalah untuk menentukan volume data dan frekuensi penulisan agar jadwal *cron job* dapat diatur secara optimal guna menghindari beban kerja berlebih pada server.

b. Tahap perancangan topologi

Merancang arsitektur *master-slave* yang menugaskan server utama (*Master*) menangani transaksi data pasien secara *real-time*, sedangkan server sekunder (*Slave*) didedikasikan untuk menerima distribusi data dan proses *backup*. Hal ini memastikan sumber daya *Master* tidak terganggu oleh proses pencadangan.

c. Tahap implementasi teknis

1) Konfigurasi replikasi: Mengaktifkan *binary logging* pada server *master* dan mengonfigurasi server *Slave* untuk menyalin perubahan data secara otomatis.
2) Integrasi mariadb-*backup*: Mengimplementasikan fitur *hot backup* agar penyalinan data dilakukan tanpa mengunci tabel basis data (*non-locking*), sehingga layanan pasien tetap aktif.
3) Otomatisasi *cron job:* Mengatur penjadwalan perintah eksekusi *backup* di Linux agar berjalan secara konsisten tanpa intervensi manual.

d. Tahap pengujian sistem

Pengujian sistem diimplementasikan dengan mensimulasikan *input* data pasien pada jaringan *hybrid* menuju server *Master*. Selanjutnya, dilakukan pemantauan terhadap status replikasi untuk memverifikasi bahwa proses penyalinan data ke server *Slave* telah berlangsung secara otomatis tanpa hambatan.

e. Tahap evaluasi dan analisis data

Pengumpulan data temporal dari setiap siklus pencadangan otomatis dilakukan untuk dianalisis menggunakan metode *Average Time*. Melalui metode ini, efektivitas durasi otomatisasi dapat diukur secara kuantitatif untuk membuktikan

konsistensi sistem dalam melakukan sinkronisasi data medis di tengah fluktuasi beban kerja jaringan *hybrid*.

3.1.2 Metode pengumpulan dan analisis data

Penelitian ini menggunakan pendekatan eksperimental dengan tujuan menganalisis efektivitas otomatisasi *backup* data pada lingkungan jaringan *hybrid* sistem rawat inap rumah sakit. Metode eksperimen dipilih karena penelitian ini melibatkan perancangan, implementasi, serta pengujian langsung terhadap sistem replikasi basis data dan mekanisme pencadangan otomatis untuk memperoleh data kinerja yang terukur dan objektif.

a. Metode replikasi *master–slave*

Metode utama yang digunakan dalam penelitian ini adalah replikasi basis data MySQL dengan arsitektur *master–slave*. Pada metode ini, server *Master* berfungsi sebagai pusat transaksi yang melayani operasi baca dan tulis data, sedangkan server *Slave* berperan sebagai server cadangan yang mereplikasi setiap perubahan data dari *Master* secara otomatis. Replikasi ini diterapkan pada lingkungan jaringan *hybrid* yang mengombinasikan infrastruktur lokal dan VPS, sehingga data pemeriksaan pasien rawat inap tetap tersinkronisasi dan tersedia meskipun terjadi gangguan pada salah satu server. Implementasi replikasi bertujuan untuk meningkatkan ketersediaan data (*availability*) serta meminimalkan risiko *single point of failure* pada sistem basis data rumah sakit.

b. Metode otomatisasi *backup*

Untuk menjamin keberlangsungan data, proses pencadangan dilakukan secara otomatis menggunakan mariadb-*backup* yang dijalankan pada server *Slave.* Pemanfaatan *hot backup* memungkinkan proses pencadangan dilakukan tanpa menghentikan layanan basis data atau mengunci tabel, sehingga tidak mengganggu aktivitas input data pasien yang berlangsung secara *real-time*. Penjadwalan *backup* dilakukan menggunakan *cron job* pada sistem operasi Linux, dengan interval waktu tertentu (misalnya setiap pukul 02.00 WIB), sehingga proses *backup* berjalan secara konsisten tanpa intervensi manual dari *administrator* sistem.

c. Metode analisis *average time*

Metode *Average Time* muncul dari kebutuhan evaluasi performa pada sistem basis data terdistribusi untuk mengukur tingkat efisiensi durasi dan stabilitas sinkronisasi secara objektif. Metode ini berlandaskan pada metodologi pengujian mekanisme replikasi yang diteliti oleh Florensia dkk. (2024, hal: 33), guna memverifikasi keandalan sistem dalam menangani beban kerja data secara terus-menerus. Dengan mengacu pada Persamaan 2.1 yang telah diuraikan pada landasan teori, data durasi yang diperoleh dari hasil pengujian akan dikalkulasi dan dianalisis menggunakan pendekatan statistik deskriptif dan komparatif sebagai berikut:

1) Analisis Efisiensi Otomatisasi Backup Data

   Analisis ini bertujuan untuk melihat stabilitas sistem dalam menangani pertumbuhan volume data dari waktu ke waktu.

2) Analisis Latensi Sinkronisasi

   Mengukur selisih waktu (latensi) antara penulisan di VPS Master hingga data muncul di VPS Slave. Nilai *Seconds_Behind_Master* akan dianalisis untuk menentukan tingkat sinkronisasi yang presisi guna menghindari keterlambatan informasi medis.

3) Analisis Performa Akses

   Menganalisis *throughput* (jumlah transaksi per detik) pada arsitektur *master-slave* yang memisahkan koneksi baca dan tulis, serta menghitung persentase peningkatan kecepatan akses data setelah beban kerja didistribusikan ke VPS Slave.

3.2 Data dan Pengumpulan Data

Tahap pengumpulan data merupakan bagian krusial dalam memastikan validitas analisis terhadap sistem otomatisasi *backup* yang diimplementasikan pada jaringan *hybrid*. Proses ini melibatkan identifikasi kebutuhan data teknis melalui studi literatur dan observasi langsung terhadap objek penelitian, yakni sistem informasi pemeriksaan pasien rawat inap. Data yang dikumpulkan mencakup data primer yang diperoleh secara *real-time* melalui hasil eksperimen pada lingkungan *Virtual Private Server* (VPS) serta data sekunder pendukung untuk memperkuat landasan teori mengenai manajemen basis data dan mekanisme replikasi. Seluruh

data yang dihimpun akan difokuskan pada pengukuran efisiensi durasi pencadangan dan konsistensi sinkronisasi objek *database* guna menjawab rumusan masalah penelitian.

3.2.1 Identifikasi jenis data

Data yang dibutuhkan dalam penelitian ini dibagi menjadi dua kategori utama guna mendukung analisis *average time* dan evaluasi sistem:

a. Data Teknis Operasional (Kuantitatif):
   1) Durasi waktu yang dibutuhkan untuk setiap proses *backup* menggunakan mariadb-*backup*.
   2) Ukuran basis data (*database size*) yang dikirimkan dari jaringan *hybrid* sistem pemeriksaan pasien rawat inap.
   3) Nilai latensi sinkronisasi atau waktu tunda (*delay*) antara server *Master* dan *Slave*.
   4) *Log* keberhasilan atau kegagalan eksekusi perintah *cron job* pada Linux.

b. Data Kebutuhan Sistem (Kualitatif/Teknis):
   1) Struktur tabel dan objek *database* sistem pemeriksaan pasien yang akan direplikasi.
   2) Konfigurasi parameter pada file **my.cnf** atau **my.ini** untuk mendukung replikasi dan otomatisasi.

3.2.2 Sumber data

Penelitian ini menggunakan dua sumber data utama untuk menjamin validitas hasil pengujian:

a. Data Primer
   1) Diperoleh langsung melalui hasil pengujian atau eksperimen pada sistem yang dibangun.
   2) Data ini mencakup catatan waktu nyata (*real-time logs*) dari proses otomatisasi *backup* dan status sinkronisasi *master-slave*.

b. Data Sekunder

1) Berasal dari studi literatur, jurnal penelitian terdahulu, dan dokumentasi resmi mengenai MySQL, mariadb-*backup*, serta sistem informasi rumah sakit.
2) Data ini digunakan sebagai pembanding atau standar referensi dalam menganalisis kinerja sistem.

3.2.3 Cara pengumpulan data

Data akan dikumpulkan melalui langkah-langkah terstruktur berikut:

a. Observasi Sistem (Eksperimen)

Peneliti melakukan pengamatan langsung terhadap jalannya simulasi pengiriman data dari jaringan *hybrid* ke server *Master*. Data primer seperti status sinkronisasi dikumpulkan dengan memantau perintah **SHOW SLAVE STATUS\G;** pada MySQL.

b. Pengujian Terkontrol (*Testing*)

Menjalankan skenario otomatisasi *backup* sesuai jadwal *cron job* yang telah ditentukan. Waktu mulai dan waktu selesai setiap sesi *backup* dicatat secara otomatis ke dalam file *log* sistem untuk diolah lebih lanjut menggunakan metode *average time*.

c. Dokumentasi *Log*

Mengambil data histori dari *log* mariadb-*backup* dan *log* Linux (*syslog/cronlog*). Dokumentasi ini berfungsi untuk memverifikasi integritas data dan konsistensi otomatisasi tanpa adanya intervensi manual.

d. Studi Literatur

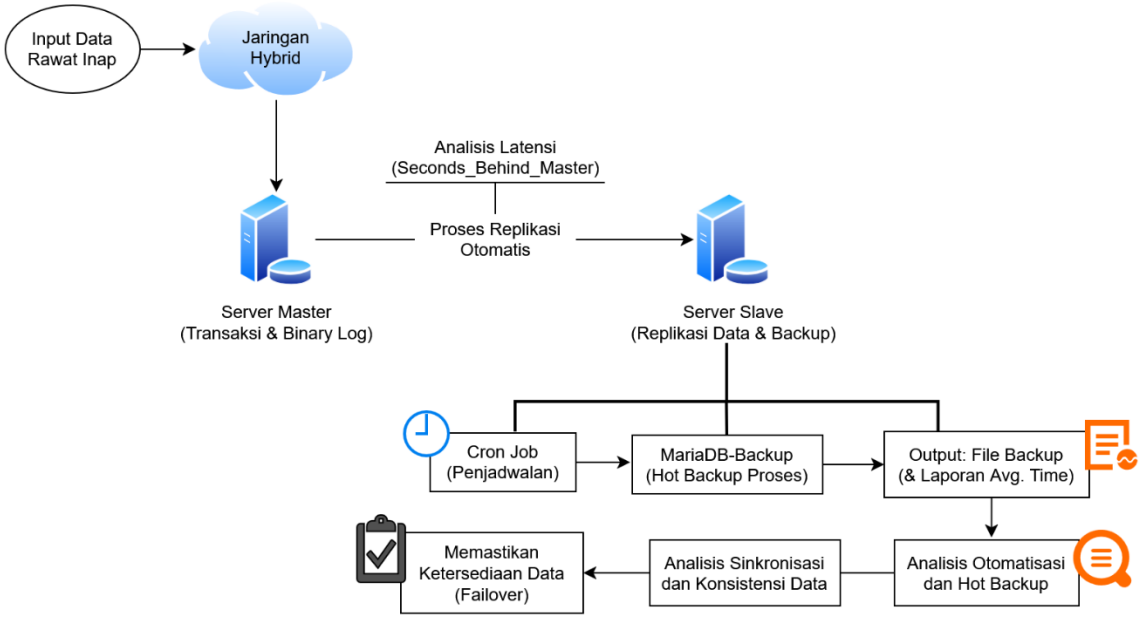
Mengumpulkan data pendukung melalui observasi pada literatur tentang sistem manajemen basis data dan sistem replikasi guna memahami bagian-bagian teknis yang harus dikonfigurasi.

3.3 Rancangan Sistem (*Hardware/Software*)

Arsitektur ini dirancang untuk beroperasi pada lingkungan jaringan *hybrid* yang menghubungkan terminal pemeriksaan pasien di unit rawat inap dengan pusat data (*data center*). Untuk rancangan sistem meliputi perangkat keras dan perangkat lunak, berikut penjelasannya:

a. Perangkat Keras (*Hardware*):
    1) Server *Master*: Server utama berbasis Linux yang berfungsi menangani transaksi data pemeriksaan pasien secara *real-time*.
    2) Server *Slave*: Server sekunder yang berfungsi sebagai replika data dan tempat dilakukannya proses otomatisasi *backup* untuk menjaga performa *Master*.
    3) Infrastruktur Jaringan: Menggunakan 2 VPS (*Virtual Private Server*) dengan spesifikasi tertentu yang sudah dikonfigurasi agar bisa saling berkomunikasi.
    4) *Client* (Terminal Rawat Inap): Perangkat komputer yang digunakan tenaga medis untuk memasukkan data pasien.
b. Perangkat Lunak (*Software*):
    1) Sistem Operasi: Linux Ubuntu Server
    2) *Database Management System:* MySQL Server dengan fitur replikasi aktif.
    3) *Backup Engine:* Mariadb-*backup* (untuk fitur *hot backup*).
    4) *Automation Tool: Cron job* (penjadwal tugas pada Linux).

Arsitektur ini mengintegrasikan jaringan *hybrid* dan otomatisasi *backup* untuk operasional rawat inap rumah sakit. Melalui replikasi *master-slave* MySQL, objek basis data disinkronkan secara otomatis guna menjamin konsistensi informasi pemeriksaan pasien rawat inap. Server *Master* melayani transaksi *real-time*, sementara server *Slave* mengelola cadangan menggunakan mariadb-*backup* dan *cron job* demi menjaga stabilitas performa server utama. Mekanisme *hot backup* ini menjamin ketersediaan data tanpa mengganggu aktivitas layanan sistem. Dapat dilihat pada gambar 3.2 berikut,



Gambar 3.2 Visualisasi Arsitektur Sistem

3.3.1 Rancangan *input*

Tahap ini menjelaskan data atau instruksi yang masuk ke dalam sistem:

a. *Input* berupa data identitas pasien, rekam medis, hasil pemeriksaan, dan tindakan medis yang dimasukkan melalui aplikasi rawat inap.
b. Parameter koneksi antara server *Master* dan *Slave* (IP *address*, *username*, *password*, dan *log-bin file*).
c. Instruksi mengenai waktu spesifik (setiap pukul 02:00 WIB) bagi sistem untuk memulai proses *backup* otomatis.

3.3.2 Rancangan proses

Menguraikan langkah-langkah teknis pengolahan data dari input menjadi cadangan yang aman:

a. Setiap data yang masuk ke server *Master* dicatat dalam *binary log* dan dikirimkan secara otomatis ke server *Slave* melalui jaringan untuk memastikan konsistensi data.
b. Kueri tulis (*read*) diarahkan ke *master*, sementara kueri baca diarahkan ke *Slave* untuk mengoptimalkan performa akses.
c. *Cron job* pada Linux secara otomatis menjalankan skrip *bash* yang memanggil perintah mariadb-*backup* sesuai jadwal.

d. Mariadb-*backup* menyalin file data fisik pada sisi *Slave* tanpa melakukan penguncian tabel (*non-blocking*), sehingga *input* data pasien pada *Master* tetap bisa berjalan secara *real-time*.

3.3.3 Rancangan *output*

Hasil akhir yang dihasilkan oleh sistem dapat dijelaskan sebagai berikut:

a. Basis data pada server *Slave* yang identik dengan *Master*, siap digunakan jika terjadi kegagalan sistem (*failover*).
b. File cadangan *database* dalam format terkompresi yang tersimpan secara aman di direktori cadangan.
c. Laporan status keberhasilan atau kegagalan proses *backup*.
d. Data numerik mengenai rata-rata waktu yang dibutuhkan sistem untuk menyelesaikan satu siklus *backup* sebagai bahan analisis performa.

3.4 Teknik Pengujian

Pengujian dilakukan dengan metode eksperimental melalui beberapa skenario simulasi beban kerja pada sistem rawat inap:

3.4.1 Pengujian sinkronisasi dan konsistensi data

Pada pengujian sinkronisasi dan konsistensi data diuraikan sebagai berikut:

a. Dilakukan input data pemeriksaan pasien secara acak pada VPS *Master* untuk memicu mekanisme replikasi.
b. Dibandingkan jumlah baris tabel dan nilai data pada VPS *Slave* segera setelah transaksi di *Master* selesai untuk memastikan konsistensi data.
c. Diamati perintah **SHOW SLAVE STATUS\G;** untuk memastikan proses *Slave_IO_Running* dan *Slave_SQL_Running* dalam kondisi aktif.

3.4.2 Pengujian otomatisasi dan *hot backup*

Untuk pengujian otomatisasi dan *hot backup* dijabarkan sebagai berikut:

a. Memverifikasi ketepatan waktu eksekusi skrip *backup* otomatis sesuai jadwal yang telah diatur dalam sistem Linux.
b. Mengukur kemampuan mariadb-*backup* dalam melakukan *hot backup* tanpa mengunci tabel (*locking*), sehingga input data pasien pada VPS *Master* tidak terhenti selama proses pencadangan berlangsung.

c. Melakukan restorasi berkala pada file *output backup* untuk memastikan data hasil otomatisasi dapat terbaca dan valid.

3.4.3 Pengujian ketersediaan (*failover*)

Rencana pengujian *failover* untuk memastikan kesinambungan layanan dijelaskan sebagai berikut:

a. Mematikan layanan MySQL pada VPS *Master* secara mendadak untuk menguji ketersediaan data pada VPS *Slave*.
b. Memastikan sistem informasi rawat inap tetap dapat melakukan kueri baca (*read*) menggunakan data di server *Slave* meskipun server utama sedang tidak tersedia.

BAB IV
HASIL DAN PEMBAHASAN

4.1 Hasil Implementasi Sistem

Bagian ini memaparkan hasil implementasi teknis yang mencakup infrastruktur jaringan *hybrid*, konfigurasi basis data, dan mekanisme otomatisasinya. Melalui dokumentasi konfigurasi, seluruh parameter teknis dianalisis untuk memastikan replikasi *master-slave* dan skema *backup* berjalan sesuai rancangan.

4.1.1 Realisasi lingkungan kerja

Pada tahap awal implementasi teknis, realisasi lingkungan kerja (*environment*) berupa infrastruktur jaringan *hybrid* yang mengintegrasikan terminal lokal rumah sakit dengan pusat data berbasis *cloud*. Lingkungan produksi (*production-grade*) ini dibangun menggunakan 2 (dua) unit *Virtual Private Server* (VPS) Linux Ubuntu Server yang berfungsi secara spesifik sebagai server *Master* dan server *Slave*. Realisasi spesifikasi perangkat keras logis (*virtual hardware*) untuk kedua server tersebut dipaparkan secara rinci pada tabel 4.1 berikut.

Tabel 4.1 Spesifikasi Teknis *Virtual Private Server* (VPS)

Komponen Sistem	Spesifikasi VPS *Master*	Spesifikasi VPS *Slave*
Sistem Operasi	Linux Ubuntu Server 22.04	Linux Ubuntu Server 22.04
Arsitektur CPU	1 Core vCPU	1 Core vCPU
Kapasitas RAM	1 GB RAM	1 GB RAM
Penyimpanan (Storage)	60 GB NVMe SSD	60 GB NVMe SSD
Konektivitas Jaringan	IP Publik Statis	IP Publik Statis

Dilakukan konfigurasi jaringan pada tingkat server agar kedua VPS dapat saling berkomunikasi secara aman melalui protokol TCP/IP.

4.1.2 Hasil konfigurasi replikasi *master-slave* MySQL

Setelah topologi jaringan *hybrid* berhasil direalisasikan, tahapan selanjutnya adalah mengonfigurasi sistem replikasi basis data *master-slave* menggunakan MariaDB/MySQL, guna memastikan setiap perubahan data rawat inap pada VPS *Master* tersinkronisasi ke VPS *Slave*.

Pada VPS *Master*, konfigurasi berkas **50-server.cnf** dilakukan dengan mengaktifkan *binary logging* (*log*_bin) dan menetapkan **server-id = 1** untuk mencatat seluruh transaksi DML (*Insert, Update, Delete*), dapat dilihat pada gambar 4.1 berikut.

```
server-id               = 1
log_bin                 = /var/log/mysql/mysql-bin.log
```

Gambar 4.1 Penetapan server-id dan *log*_bin *Node Master*

Pada VPS *Slave*, ditetapkan **server-id = 2** dengan pointer replikasi yang diarahkan ke IP publik statis VPS *Master* menggunakan kredensial khusus, dapat dilihat pada gambar 4.2 berikut.

```
server-id               = 2
log_bin                 = /var/log/mysql/mysql-bin.log
```

Gambar 4.2 Penetapan server-id dan *log*_bin *Node Slave*

Setelah konfigurasi file **50-server.cnf** dilakukan pada masing-masing *node*, selanjutnya masuk ke MariaDB pada sisi *node master* untuk memberikan akses replikasi pada *node slave* melalui contoh baris perintah **GRANT REPLICATION SLAVE ON *.* TO 'user'@'IP Slave' IDENTIFIED BY 'password'**.

Untuk melihat informasi status dari replikasi *node master* dengan memasukkan baris perintah **SHOW MASTER STATUS;** untuk melihat nama file *binary log* dan posisi *log* replikasi yang akan dimasukkan pada sisi replikasi *node slave* agar proses sinkronisasi antar database berjalan akurat dari titik koordinat tersebut, dapat dilihat pada gambar 4.3 berikut.

```
MariaDB [(none)]> SHOW MASTER STATUS;
+------------------+----------+--------------+------------------+
| File             | Position | Binlog_Do_DB | Binlog_Ignore_DB |
+------------------+----------+--------------+------------------+
| mysql-bin.000003 |   181952 |              | phpmyadmin       |
+------------------+----------+--------------+------------------+
1 row in set (0.002 sec)
```

Gambar 4.3 Status Replikasi pada *Node Master*

Ketika informasi sudah didapatkan dari status *node master* tersebut, selanjutnya masuk ke MariaDB pada sisi *node slave* untuk melakukan konfigurasi sinkronisasi replikasi dari informasi file *binary log* dan posisi *log* yang akan digunakan, contoh baris sintaksnya dapat dilihat pada gambar 4.4 berikut

```
change master to
   -> master_host='Alamat IP Master',
   -> master_user='user',
   -> master_password='password',
   -> master_port=3306,
   -> master_log_file='mysql-bin.000003',
   -> master_log_pos=181952;
```

Gambar 4.4 Baris Sintaks Konfigurasi Replikasi *Node Slave*

Keberhasilan replikasi diverifikasi melalui kueri **SHOW SLAVE STATUS\G;** dengan hasil parameter kritis disajikan pada tabel 4.2 berikut.

Tabel 4.2 Parameter Kritis Status Replikasi pada *Node Slave*

Parameter Replikasi	**Nilai Status (*Output*)**	**Indikasi Sistem**
Slave_IO_Running	**Yes**	Terhubung ke Master dan membaca *Binary Log*
Slave_SQL_Running	**Yes**	Berhasil mengeksekusi kueri dari *Master* ke Database *Slave*
Master_Log_File	mysql-bin.000003	Berkas *log* aktif yang sedang dibaca saat ini
Read_Master_Log_Pos	181952	Posisi pointer pembacaan baris *log* terakhir

Nilai *Yes* pada parameter *Slave_IO_Running* dan *Slave_SQL_Running* di tabel 4.2 membuktikan bahwa *Thread* I/O *node slave* berhasil terhubung ke *node master*, membaca *binary log*, dan menuliskan instruksi data ke penyimpanan lokal tanpa error.

4.1.3 Hasil implementasi otomatisasi *backup* terjadwal

Setelah replikasi *master-slave* dipastikan berjalan stabil, kemudian diimplementasikan sistem otomatisasi backup pada VPS *slave* dengan mengintegrasikan mariadb-*backup* sebagai mesin pencadangan utama dan *cron job* Linux Ubuntu Server sebagai penjadwal tugas. Dua skrip bash disusun untuk keperluan tersebut, yakni mysql_*full_backup*.sh untuk pencadangan penuh dan mysql_*incremental_backup*.sh untuk pencadangan inkremental, keduanya memanfaatkan kemampuan *hot backup* mariadb-*backup* yang dapat menyalin berkas data fisik tanpa penguncian tabel, untuk kedua skripnya dapat dilihat pada gambar 4.5 dan gambar 4.6 berikut.

```
DATE=$(date +%F)
BASE_DIR="/backup/mysql/${DATE}"
FULL_DIR="${BASE_DIR}/full"
LOG_FILE="/var/log/mariadb_backup.log"

log() {
    echo "[$(date '+%Y-%m-%d %H:%M:%S')] $1" >> "$LOG_FILE"
}

log "====== MULAI FULL BACKUP ======"

# Buat direktori
mkdir -p "$FULL_DIR"

# Jalankan full backup
if mariabackup --backup \
    --target-dir="$FULL_DIR" \
    >> "$LOG_FILE" 2>&1; then

    # Prepare full backup agar siap jadi base incremental
    mariabackup --prepare \
        --apply-log-only \
        --target-dir="$FULL_DIR" \
        >> "$LOG_FILE" 2>&1

    log "Full backup BERHASIL: $FULL_DIR"
else
    log "Full backup GAGAL!"
    exit 1
fi

# Hapus backup lebih dari 7 hari
find /backup/mysql -mindepth 1 -maxdepth 1 -type d -mtime +7 -exec rm -rf {} \;
log "Cleanup backup lama selesai"
log "====== SELESAI FULL BACKUP ======"
```

Gambar 4.5 Skrip mysql_full_backup.sh

Sementara itu, skrip kedua yang dirancang secara spesifik untuk menangani proses penyalinan perubahan data secara inkremental disajikan pada gambar 4.6 berikut.

```
DATE=$(date +%F)
HOUR=$(date +%H)
BASE_DIR="/backup/mysql/${DATE}"
FULL_DIR="${BASE_DIR}/full"
LOG_FILE="/var/log/mariadb_backup.log"

log() {
    echo "[$(date '+%Y-%m-%d %H:%M:%S')] $1" >> "$LOG_FILE"
}

log "====== MULAI INCREMENTAL BACKUP (jam ${HOUR}) ======"

# Tentukan nama folder target berdasarkan jam
case "$HOUR" in
    09) TARGET_DIR="${BASE_DIR}/inc-09"
        ;;
    15) TARGET_DIR="${BASE_DIR}/inc-15"
        ;;
    21) TARGET_DIR="${BASE_DIR}/inc-21"
        ;;
    *)  log "Jam tidak dikenal: $HOUR. Backup dibatalkan."
        exit 1
        ;;
esac

# Tentukan base incremental (dari mana perubahan dihitung)
# Urutan prioritas: inc-15 > inc-09 > full
if   [ -d "${BASE_DIR}/inc-15" ] && [ "$HOUR" == "21" ]; then
    BASE_INC="${BASE_DIR}/inc-15"
elif [ -d "${BASE_DIR}/inc-09" ] && [ "$HOUR" == "15" ]; then
    BASE_INC="${BASE_DIR}/inc-09"
else
    BASE_INC="$FULL_DIR"
fi
# Validasi: full backup harus sudah ada
if [ ! -d "$FULL_DIR" ]; then
    log "ERROR: Full backup tidak ditemukan di $FULL_DIR"
    log "Jalankan full backup terlebih dahulu!"
    exit 1
fi

# Buat direktori target
mkdir -p "$TARGET_DIR"

log "Base incremental: $BASE_INC"
log "Target: $TARGET_DIR"

# Jalankan incremental backup
if mariabackup --backup \
    --target-dir="$TARGET_DIR" \
    --incremental-basedir="$BASE_INC" \
    >> "$LOG_FILE" 2>&1; then

    log "Incremental backup BERHASIL: $TARGET_DIR"
else
    log "Incremental backup GAGAL!"
    exit 1
fi

log "====== SELESAI INCREMENTAL BACKUP (jam ${HOUR}) ======"
```

Gambar 4.6 Skrip mysql_incremental_backup.sh

Seluruh hasil cadangan disimpan pada direktori /*backup*/mysql dan dijadwalkan melalui crontab -e, *full backup* dieksekusi setiap pukul 02.00 WIB pada periode lalu lintas data terendah, sedangkan *incremental backup* dijalankan pada pukul 09.00, 15.00, dan 21.00 WIB. Konfigurasi baris perintah *crontab* yang berhasil diimplementasikan dipaparkan pada tabel 4.3 berikut.

Tabel 4.3 Konfigurasi Penjadwalan *Crontab* pada Linux *Node Slave*

No	Sintaks *Crontab*	Representasi Waktu	Aksi / Skrip yang Dieksekusi	Jenis Operasi Backup
1	0 2 * * *	Setiap hari pukul 02:00 WIB	/usr/local/bin/mysql_full_backup.sh	*Full backup* (Salinan Fisik Basis Data Utuh)
2	0 9 * * *	Setiap hari pukul 09:00 WIB	/usr/local/bin/mysql_incremental_backup.sh	*Incremental backup* (Salinan Perubahan Data)
3	0 15 * * *	Setiap hari pukul 15:00 WIB	/usr/local/bin/mysql_incremental_backup.sh	*Incremental backup* (Salinan Perubahan Data)
4	0 21 * * *	Setiap hari pukul 21:00 WIB	/usr/local/bin/mysql_incremental_backup.sh	*Incremental backup* (Salinan Perubahan Data)

Berkas cadangan yang dihasilkan memiliki penamaan terstruktur berbasis tanggal (*timestamping*), yang mempermudah proses identifikasi apabila sewaktu-waktu diperlukan pemulihan data (*data recovery*), contohnya dapat dilihat pada gambar 4.7 dan gambar 4.8 berikut.

```
[redacted]# ls -lh /backup/mysql
total 36K
drwxr-xr-x 6 root root 4.0K Jun  6 21:00 2026-06-06
drwxr-xr-x 6 root root 4.0K Jun  7 21:00 2026-06-07
drwxr-xr-x 6 root root 4.0K Jun  8 21:00 2026-06-08
drwxr-xr-x 6 root root 4.0K Jun  9 21:00 2026-06-09
drwxr-xr-x 6 root root 4.0K Jun 10 21:00 2026-06-10
drwxr-xr-x 6 root root 4.0K Jun 11 21:00 2026-06-11
drwxr-xr-x 6 root root 4.0K Jun 12 21:00 2026-06-12
drwxr-xr-x 6 root root 4.0K Jun 13 21:00 2026-06-13
drwxr-xr-x 5 root root 4.0K Jun 14 15:00 2026-06-14
```

Gambar 4.7 Direktori Hasil *Backup* pada *Node Slave*

Adapun rincian struktur dan isi berkas yang terdapat di dalam salah satu direktori pencadangan harian tersebut dapat dilihat secara lebih jelas pada gambar 4.8 berikut.

```
[redacted] ls -lh /backup/mysql/2026-06-14
total 16K
drwxr-xr-x 9 root root 4.0K Jun 14 02:00 full
drwxr-xr-x 9 root root 4.0K Jun 14 09:00 inc-09
drwxr-xr-x 9 root root 4.0K Jun 14 15:00 inc-15
drwxr-xr-x 9 root root 4.0K Jun 14 21:00 inc-21
[redacted] ls -lh /backup/mysql/2026-06-14/full
total 125M
-rw-r----- 1 root root 416K Jun 14 02:00 aria_log.00000001
-rw-r----- 1 root root   52 Jun 14 02:00 aria_log_control
-rw-r----- 1 root root  285 Jun 14 02:00 backup-my.cnf
-rw-r----- 1 root root  49M Jun 14 02:00 ib_logfile0
-rw-r----- 1 root root  76M Jun 14 02:00 ibdata1
drwx------ 2 root root 4.0K Jun 14 02:00 monitor
drwx------ 2 root root 4.0K Jun 14 02:00 mysql
drwx------ 2 root root 4.0K Jun 14 02:00 performance_schema
drwx------ 2 root root 4.0K Jun 14 02:00 phpmyadmin
drwx------ 2 root root 4.0K Jun 14 02:00 politeknik
drwx------ 2 root root 4.0K Jun 14 02:00 rawat_inap_vps
drwx------ 2 root root  12K Jun 14 02:00 sys
-rw-r----- 1 root root   35 Jun 14 02:00 xtrabackup_binlog_info
-rw-r----- 1 root root  107 Jun 14 02:00 xtrabackup_checkpoints
-rw-r----- 1 root root  543 Jun 14 02:00 xtrabackup_info
```

Gambar 4.8 Isi dari Folder Hasil *Backup* pada 14 Juni 2026

Dengan keberhasilan implementasi otomatisasi *backup* terjadwal ini, telah mewujudkan mekanisme pencadangan data rekam medis pasien rawat inap yang berjalan di latar belakang (*background process*) secara konsisten, efisien, dan selaras dengan standar tata kelola infrastruktur data rumah sakit.

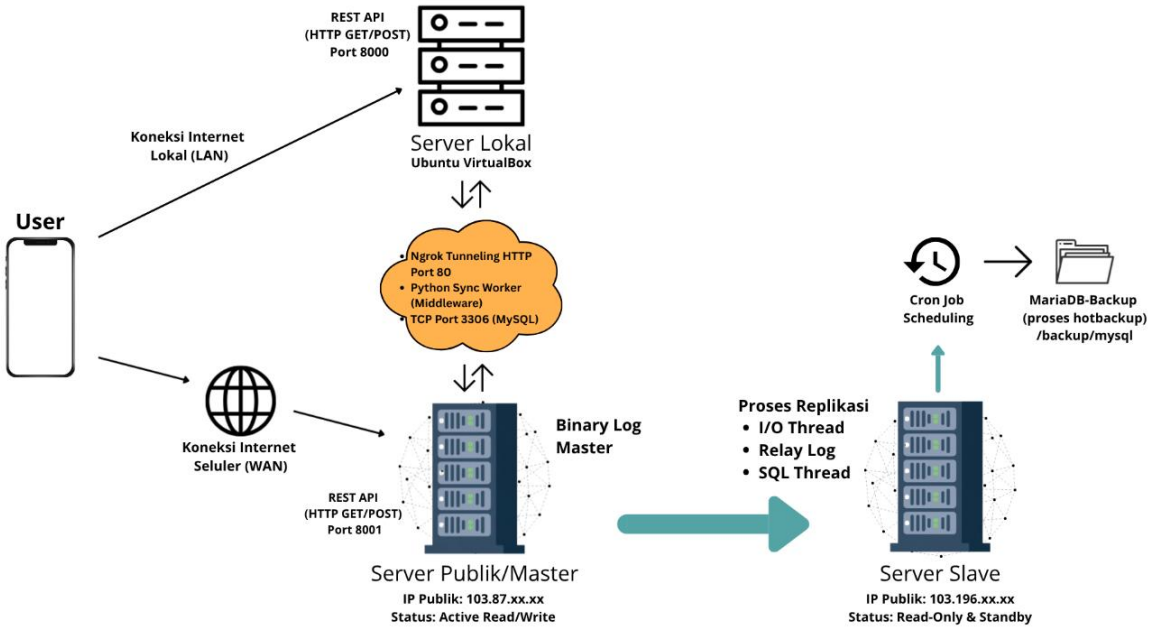
4.2 Pengujian dan Analisis Data

Subbab ini menyajikan hasil pengujian dan analisis performa serta keandalan sistem otomatisasi *backup* data rekam medis. Melalui injeksi data pada jaringan *hybrid*, evaluasi difokuskan pada efektivitas replikasi *master-slave* MySQL,

efisiensi mariadb-*backup*, dan ketepatan *cron job*. Analisis ini bertujuan memastikan sinkronisasi, konsistensi, dan integritas data antara *node master* dan *node slave* terjaga secara optimal.

4.2.1 Arsitektur jaringan *hybrid* dan *master-slave*

Bagian ini menyajikan rancangan arsitektur sistem yang mengintegrasikan infrastruktur jaringan *hybrid* dengan topologi basis data *master-slave*. Penjelasan ini menjadi landasan teknis untuk memahami jalur transmisi data rekam medis dari terminal lokal menuju *node master*, hingga proses propagasi otomatis ke *node slave* sebagai pelaksana cadangan data harian, dilihat pada gambar 4.9 berikut.



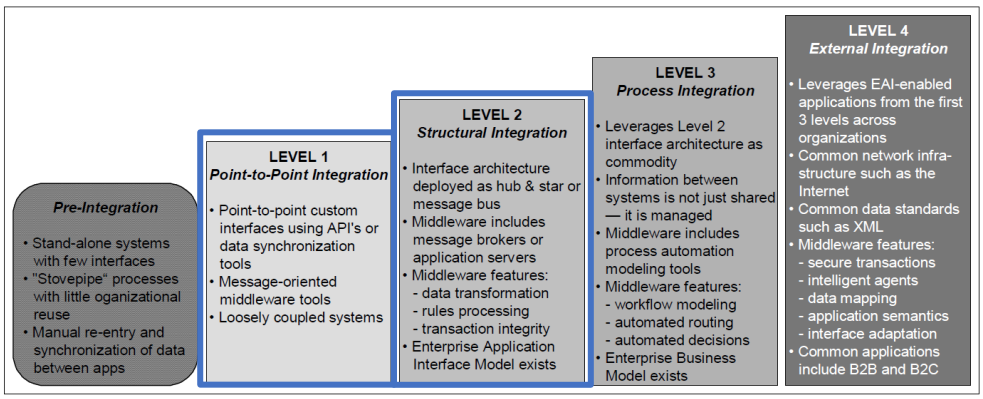
Gambar 4.9 Alur Sistem Jaringan *Hybrid* dan *master-slave*

Berikut adalah penjelasan mengenai alur operasional sistem jaringan *hybrid* dan arsitektur *master-slave* sesuai dengan visualisasi arsitektur gambar 4.8 tersebut:

a. Pengguna mengakses aplikasi melalui jalur lokal (LAN/Port 8000) atau publik (WAN/Port 8001), yang keduanya terintegrasi ke dalam jaringan *hybrid* menggunakan Ngrok *Tunneling* dan *Python Sync Worker* untuk sinkronisasi basis data (MySQL/Port 3306).
b. Transaksi *read/write* diproses secara *real-time* pada *node master* melalui *Binary Log*, lalu direplikasi secara otomatis oleh *node slave* melalui mekanisme *I/O Thread*, *Relay Log*, dan *SQL Thread*.

c. Tugas *hot backup* diisolasi sepenuhnya pada *node slave* menggunakan *cron job* dan mariadb-*backup* untuk menghasilkan berkas cadangan otomatis di dalam direktori /*backup*/mysql.

Evaluasi kematangan infrastruktur dalam mengintegrasikan sistem basis data dapat diukur menggunakan instrumen *Integration Maturity Model* (Model Kematangan Integrasi) gagasan Schmidt. Kerangka kerja ini mengklasifikasikan evolusi sistem secara hierarkis, mulai dari fase praintegrasi yang terisolasi hingga tahap integrasi eksternal antar-organisasi. Pemetaan tersebut menjadi panduan strategis untuk menilai stabilitas operasional arsitektur terdistribusi, termasuk skema replikasi pada jaringan *hybrid*. Visualisasi tahapan model kematangan integrasi aplikasi ini diilustrasikan pada gambar 4.10 berikut.



Gambar 4.10 Model Kematangan Schmidt untuk Integrasi Aplikasi
(Sumber: Mendoza dkk., 2003, hal: 2)

Berdasarkan 4 level *Integration Maturity Model* Schmidt pada tahun 2000 seperti pada gambar 4.10 diatas, arsitektur replikasi *master-slave* yang dibangun secara fungsional memenuhi karakteristik **Level 1: *Point-to-Point Integration*** melalui distribusi data langsung via propagasi *binary log*. Namun, capaian tingkat data yang konsisten dan jaminan integritas transaksi antar server di lingkungan *cloud* membuktikan bahwa infrastruktur jaringan *hybrid* ini telah meletakkan landasan kuat menuju **Level 2: *Structural Integration***. Keberhasilan otomatisasi mariadb-*backup* dan *cron job* ini memastikan rumah sakit memiliki infrastruktur teknologi yang tangguh sebagai salah satu Faktor Penentu Keberhasilan (*Critical Success Factors*) strategis integrasi sistem (Mendoza dkk., 2003, hal: 3).

4.2.2 Pengujian sinkronisasi dan konsistensi data pada replikasi

Pengujian tersebut disimulasikan dengan menginjeksi minimal **50 baris data (*records*)** pemeriksaan pasien rawat inap dari jaringan *hybrid* ke dalam basis data utama pada *node master*. Segera setelah proses injeksi selesai, dilakukan verifikasi silang (*cross-validation*) secara langsung pada kedua server. Langkah pengujian ini dilakukan melalui dua tahapan pembuktian:

a. Memastikan parameter *Slave_IO_Running* dan *Slave_SQL_Running* tetap berada pada status *Yes* selama proses transaksi berlangsung, guna membuktikan tidak adanya kegagalan pipa komunikasi (*replication pipeline error*) akibat beban penulisan data kesehatan yang masuk.
b. Mengeksekusi kueri agregasi **SELECT COUNT(*)** serta memeriksa kesamaan nilai *Primary Key* pada tabel-tabel kritis database pemeriksaan pasien rawat inap antara *node master* dan *node slave*.

Hasil komparasi struktural dan jumlah baris data yang diperoleh dari arsitektur terdistribusi ini disajikan pada tabel 4.4 berikut.

Tabel 4.4 Komparasi Jumlah Baris dan Konsistensi Objek Basis Data

No	Nama Tabel Basis Data	Jumlah Baris (Master)	Jumlah Baris (Slave)	Selisih (Delta)	Status Konsistensi
1	**pasien**	50	50	0	Konsisten
2	**users**	6	6	0	Konsisten
3	**visit**	50	50	0	Konsisten

Berdasarkan observasi pada tabel 4.4, hasil pengujian menunjukkan nilai selisih (*delta*) sebesar 0 (nol) pada seluruh tabel yang diamati. Hal ini membuktikan bahwa setiap paket data rekam medis pasien yang dituliskan pada *node master* telah diteruskan oleh *Binary Log* dan dieksekusi kembali secara sempurna oleh *SQL Thread* pada *node slave,* dapat dilihat pada gambar 4.11 dan gambar 4.12 berikut.

```
MariaDB [rawat_inap_vps]> SELECT COUNT(*) FROM pasien;
+----------+
| COUNT(*) |
+----------+
|       50 |
+----------+
1 row in set (0.002 sec)

MariaDB [rawat_inap_vps]> SELECT COUNT(*) FROM users;
+----------+
| COUNT(*) |
+----------+
|        6 |
+----------+
1 row in set (0.001 sec)

MariaDB [rawat_inap_vps]> SELECT COUNT(*) FROM visit;
+----------+
| COUNT(*) |
+----------+
|       50 |
+----------+
1 row in set (0.001 sec)
```

Gambar 4.11 Kueri Validasi Baris Data pada *Node Master*

Sebagai bentuk verifikasi silang terhadap kueri peladen utama di atas, hasil eksekusi kueri untuk memvalidasi kesamaan jumlah baris data yang telah direplikasi pada *node slave* disajikan pada gambar 4.12 berikut.

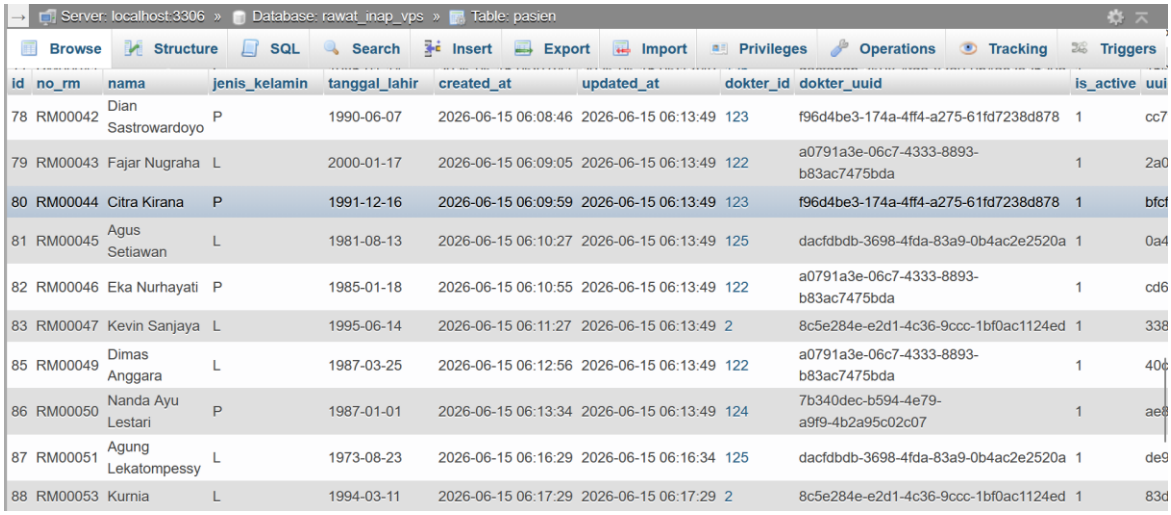
```
MariaDB [rawat_inap_vps]> SELECT COUNT(*) FROM pasien;
+----------+
| COUNT(*) |
+----------+
|       50 |
+----------+
1 row in set (0.051 sec)

MariaDB [rawat_inap_vps]> SELECT COUNT(*) FROM users;
+----------+
| COUNT(*) |
+----------+
|        6 |
+----------+
1 row in set (0.006 sec)

MariaDB [rawat_inap_vps]> SELECT COUNT(*) FROM visit;
+----------+
| COUNT(*) |
+----------+
|       50 |
+----------+
1 row in set (0.001 sec)
```

Gambar 4.12 Kueri Validasi Baris Data pada *Node Slave*

Berikut ini merupakan beberapa potongan data yang di replikasi oleh *master* dan *slave*, disajikan menggunakan baris tabel phpMyAdmin untuk mempermudah penyajian data yang beragam.



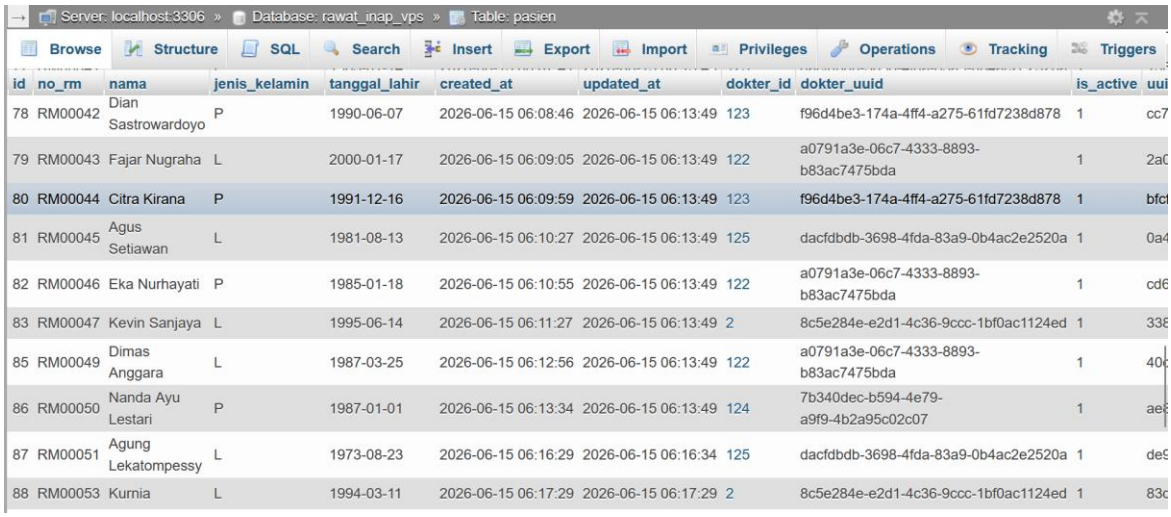
Server: localhost:3306 » Database: rawat_inap_vps » Table: pasien

Browse | Structure | SQL | Search | Insert | Export | Import | Privileges | Operations | Tracking | Triggers

id	no_rm	nama	jenis_kelamin	tanggal_lahir	created_at	updated_at	dokter_id	dokter_uuid	is_active	uui
78	RM00042	Dian Sastrowardoyo	P	1990-06-07	2026-06-15 06:08:46	2026-06-15 06:13:49	123	f96d4be3-174a-4ff4-a275-61fd7238d878	1	cc7
79	RM00043	Fajar Nugraha	L	2000-01-17	2026-06-15 06:09:05	2026-06-15 06:13:49	122	a0791a3e-06c7-4333-8893-b83ac7475bda	1	2a0
80	RM00044	Citra Kirana	P	1991-12-16	2026-06-15 06:09:59	2026-06-15 06:13:49	123	f96d4be3-174a-4ff4-a275-61fd7238d878	1	bfcf
81	RM00045	Agus Setiawan	L	1981-08-13	2026-06-15 06:10:27	2026-06-15 06:13:49	125	dacfdbdb-3698-4fda-83a9-0b4ac2e2520a	1	0a4
82	RM00046	Eka Nurhayati	P	1985-01-18	2026-06-15 06:10:55	2026-06-15 06:13:49	122	a0791a3e-06c7-4333-8893-b83ac7475bda	1	cd6
83	RM00047	Kevin Sanjaya	L	1995-06-14	2026-06-15 06:11:27	2026-06-15 06:13:49	2	8c5e284e-e2d1-4c36-9ccc-1bf0ac1124ed	1	338
85	RM00049	Dimas Anggara	L	1987-03-25	2026-06-15 06:12:56	2026-06-15 06:13:49	122	a0791a3e-06c7-4333-8893-b83ac7475bda	1	40¢
86	RM00050	Nanda Ayu Lestari	P	1987-01-01	2026-06-15 06:13:34	2026-06-15 06:13:49	124	7b340dec-b594-4e79-a9f9-4b2a95c02c07	1	ae8
87	RM00051	Agung Lekatompessy	L	1973-08-23	2026-06-15 06:16:29	2026-06-15 06:16:34	125	dacfdbdb-3698-4fda-83a9-0b4ac2e2520a	1	de9
88	RM00053	Kurnia	L	1994-03-11	2026-06-15 06:17:29	2026-06-15 06:17:29	2	8c5e284e-e2d1-4c36-9ccc-1bf0ac1124ed	1	83d

Gambar 4.13 Potongan Baris Data pada Tabel pasien *Node Master*

Dilihat pada gambar 4.13 diatas menyajikan tampilan antarmuka pengelolaan basis data MySQL yang memuat contoh rincian rekaman identitas medis pasien pada tabel **pasien** sisi *node master* di dalam sistem rawat inap.



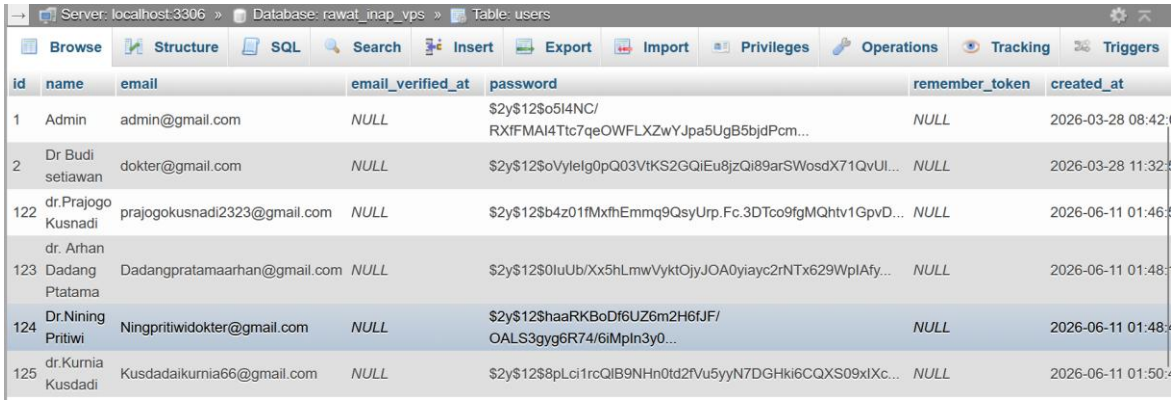
Server: localhost:3306 » Database: rawat_inap_vps » Table: pasien

Browse | Structure | SQL | Search | Insert | Export | Import | Privileges | Operations | Tracking | Triggers

id	no_rm	nama	jenis_kelamin	tanggal_lahir	created_at	updated_at	dokter_id	dokter_uuid	is_active	uui
78	RM00042	Dian Sastrowardoyo	P	1990-06-07	2026-06-15 06:08:46	2026-06-15 06:13:49	123	f96d4be3-174a-4ff4-a275-61fd7238d878	1	cc7
79	RM00043	Fajar Nugraha	L	2000-01-17	2026-06-15 06:09:05	2026-06-15 06:13:49	122	a0791a3e-06c7-4333-8893-b83ac7475bda	1	2a0
80	RM00044	Citra Kirana	P	1991-12-16	2026-06-15 06:09:59	2026-06-15 06:13:49	123	f96d4be3-174a-4ff4-a275-61fd7238d878	1	bfcf
81	RM00045	Agus Setiawan	L	1981-08-13	2026-06-15 06:10:27	2026-06-15 06:13:49	125	dacfdbdb-3698-4fda-83a9-0b4ac2e2520a	1	0a4
82	RM00046	Eka Nurhayati	P	1985-01-18	2026-06-15 06:10:55	2026-06-15 06:13:49	122	a0791a3e-06c7-4333-8893-b83ac7475bda	1	cd6
83	RM00047	Kevin Sanjaya	L	1995-06-14	2026-06-15 06:11:27	2026-06-15 06:13:49	2	8c5e284e-e2d1-4c36-9ccc-1bf0ac1124ed	1	338
85	RM00049	Dimas Anggara	L	1987-03-25	2026-06-15 06:12:56	2026-06-15 06:13:49	122	a0791a3e-06c7-4333-8893-b83ac7475bda	1	40¢
86	RM00050	Nanda Ayu Lestari	P	1987-01-01	2026-06-15 06:13:34	2026-06-15 06:13:49	124	7b340dec-b594-4e79-a9f9-4b2a95c02c07	1	ae8
87	RM00051	Agung Lekatompessy	L	1973-08-23	2026-06-15 06:16:29	2026-06-15 06:16:34	125	dacfdbdb-3698-4fda-83a9-0b4ac2e2520a	1	de9
88	RM00053	Kurnia	L	1994-03-11	2026-06-15 06:17:29	2026-06-15 06:17:29	2	8c5e284e-e2d1-4c36-9ccc-1bf0ac1124ed	1	83c

Gambar 4.14 Potongan Baris Data pada Tabel pasien *Node Slave*

Dilihat pada gambar 4.14 diatas menyajikan tampilan antarmuka pengelolaan basis data MySQL yang memuat contoh rincian rekaman identitas medis pasien pada tabel **pasien** sisi *node slave* di dalam sistem rawat inap.



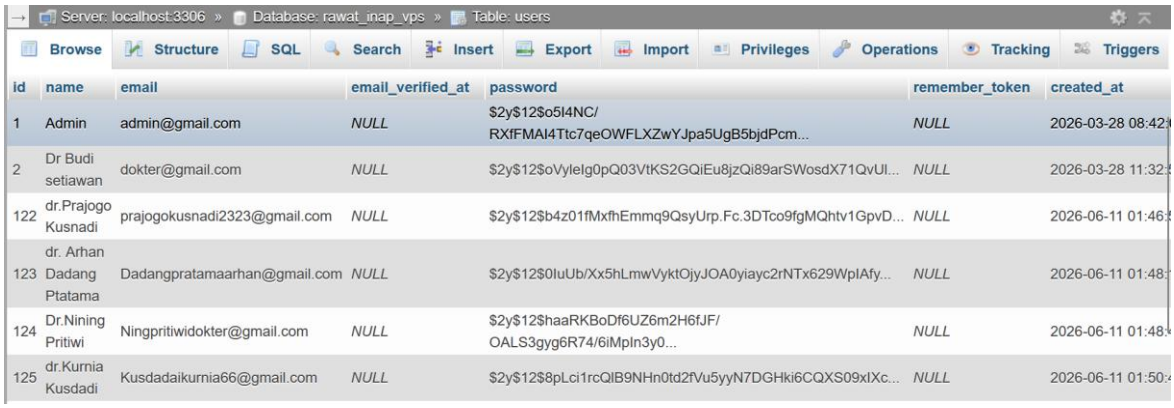
Server: localhost:3306 » Database: rawat_inap_vps » Table: users

Browse | Structure | SQL | Search | Insert | Export | Import | Privileges | Operations | Tracking | Triggers

id	name	email	email_verified_at	password	remember_token	created_at
1	Admin	admin@gmail.com	NULL	$2y$12$o5l4NC/ RXfFMAl4Ttc7qeOWFLXZwYJpa5UgB5bjdPcm...	NULL	2026-03-28 08:42:
2	Dr Budi setiawan	dokter@gmail.com	NULL	$2y$12$oVyIeIg0pQ03VtKS2GQiEu8jzQi89arSWosdX71QvUl...	NULL	2026-03-28 11:32:
122	dr.Prajogo Kusnadi	prajogokusnadi2323@gmail.com	NULL	$2y$12$b4z01fMxfhEmmq9QsyUrp.Fc.3DTco9fgMQhtv1GpvD...	NULL	2026-06-11 01:46:
123	dr. Arhan Dadang Ptatama	Dadangpratamaarhan@gmail.com	NULL	$2y$12$0IuUb/Xx5hLmwVyktOjyJOA0yiayc2rNTx629WpIAfy...	NULL	2026-06-11 01:48:
124	Dr.Nining Pritiwi	Ningpritiwidokter@gmail.com	NULL	$2y$12$haaRKBoDf6UZ6m2H6fJF/ OALS3gyg6R74/6iMpIn3y0...	NULL	2026-06-11 01:48:
125	dr.Kurnia Kusdadi	Kusdadaikurnia66@gmail.com	NULL	$2y$12$8pLci1rcQlB9NHn0td2fVu5yyN7DGHki6CQXS09xIXc...	NULL	2026-06-11 01:50:

Gambar 4.15 Potongan Baris Data pada Tabel users *Node Master*

Dapat diamati pada gambar 4.15 diatas menyajikan tampilan antarmuka pengelolaan basis data MySQL yang memuat contoh rincian identitas *login* dokter dan *administrator* pada tabel **users** sisi *node master* di dalam sistem rawat inap.



Server: localhost:3306 » Database: rawat_inap_vps » Table: users

Browse | Structure | SQL | Search | Insert | Export | Import | Privileges | Operations | Tracking | Triggers

id	name	email	email_verified_at	password	remember_token	created_at
1	Admin	admin@gmail.com	NULL	$2y$12$o5l4NC/ RXfFMAl4Ttc7qeOWFLXZwYJpa5UgB5bjdPcm...	NULL	2026-03-28 08:42:
2	Dr Budi setiawan	dokter@gmail.com	NULL	$2y$12$oVyIeIg0pQ03VtKS2GQiEu8jzQi89arSWosdX71QvUl...	NULL	2026-03-28 11:32:
122	dr.Prajogo Kusnadi	prajogokusnadi2323@gmail.com	NULL	$2y$12$b4z01fMxfhEmmq9QsyUrp.Fc.3DTco9fgMQhtv1GpvD...	NULL	2026-06-11 01:46:
123	dr. Arhan Dadang Ptatama	Dadangpratamaarhan@gmail.com	NULL	$2y$12$0IuUb/Xx5hLmwVyktOjyJOA0yiayc2rNTx629WpIAfy...	NULL	2026-06-11 01:48:
124	Dr.Nining Pritiwi	Ningpritiwidokter@gmail.com	NULL	$2y$12$haaRKBoDf6UZ6m2H6fJF/ OALS3gyg6R74/6iMpIn3y0...	NULL	2026-06-11 01:48:
125	dr.Kurnia Kusdadi	Kusdadaikurnia66@gmail.com	NULL	$2y$12$8pLci1rcQlB9NHn0td2fVu5yyN7DGHki6CQXS09xIXc...	NULL	2026-06-11 01:50:

Gambar 4.16 Potongan Baris Data pada Tabel users *Node Slave*

Dilihat pada gambar 4.16 diatas menyajikan tampilan antarmuka pengelolaan basis data MySQL yang memuat contoh rincian identitas *login* dokter dan *administrator* pada tabel **users** sisi *node slave* di dalam sistem rawat inap.


Server: localhost:3306 » Database: rawat_inap_vps » Table: visit

Browse | Structure | SQL | Search | Insert | Export | Import | Privileges | Operations | Tracking | Triggers

id	pasien_id	keluhan	pemeriksaan_fisik	diagnosa	tindakan	created_at	updated_at	uuid
75	67	demam, pilek dan badan terasa pegal pegal	NULL	influenza ringan	terapi simptomatik dan istirahat yang cukup	2026-06-15 09:11:18	2026-06-15 09:11:18	673fe931-57d7-4fb5-a5c9-e8243b44c80c
76	77	pegal diseluruh tubuh setelah aktivitas berat	NULL	Myalgia	pemberian analgesik dan istirahat	2026-06-15 09:11:59	2026-06-15 09:11:59	ba5c675b-4683-4579-90d4-522aab8f6e5
77	81	kontrol tekanan darah bulanan	NULL	Hipertensi ringan	evaluasi terapi dan edukasi untuk pola hidup sehat	2026-06-15 09:12:45	2026-06-15 09:12:45	2805ed7e-c7f6-4e76-904b-1fe914c885e9
78	87	badan lemas dan nafsu makan menurun	NULL	fatigue ringan	pemberian vitamin dan edukasi untuk menjaga pola hidup sehat	2026-06-15 09:13:26	2026-06-15 09:13:26	41b9e40f-b248-4d47-8e1f-e0228ba49141

Gambar 4.17 Potongan Baris Data pada Tabel visit *Node Master*

Pada gambar 4.17 diatas menyajikan tampilan antarmuka pengelolaan basis data MySQL yang memuat contoh rincian rekaman identitas medis pasien pada tabel **visit** sisi *node master* di dalam sistem rawat inap.


Server: localhost:3306 » Database: rawat_inap_vps » Table: visit

Browse | Structure | SQL | Search | Insert | Export | Import | Privileges | Operations | Tracking | Triggers

id	pasien_id	keluhan	pemeriksaan_fisik	diagnosa	tindakan	created_at	updated_at	uuid
75	67	demam, pilek dan badan terasa pegal pegal	NULL	influenza ringan	terapi simptomatik dan istirahat yang cukup	2026-06-15 09:11:18	2026-06-15 09:11:18	673fe931-57d7-4fb5-a5c9-e8243b44c80c
76	77	pegal diseluruh tubuh setelah aktivitas berat	NULL	Myalgia	pemberian analgesik dan istirahat	2026-06-15 09:11:59	2026-06-15 09:11:59	ba5c675b-4683-4579-90d4-522aab8f6e5
77	81	kontrol tekanan darah bulanan	NULL	Hipertensi ringan	evaluasi terapi dan edukasi untuk pola hidup sehat	2026-06-15 09:12:45	2026-06-15 09:12:45	2805ed7e-c7f6-4e76-904b-1fe914c885e9
78	87	badan lemas dan nafsu makan menurun	NULL	fatigue ringan	pemberian vitamin dan edukasi untuk menjaga pola hidup sehat	2026-06-15 09:13:26	2026-06-15 09:13:26	41b9e40f-b248-4d47-8e1f-e0228ba49141

Gambar 4.18 Potongan Baris Data pada Tabel visit *Node Slave*

Dilihat pada gambar 4.18 diatas menyajikan tampilan antarmuka pengelolaan basis data MySQL yang memuat contoh rincian rekaman identitas medis pasien pada tabel **visit** sisi *node slave* di dalam sistem rawat inap.

Dapat dilihat dari gambar 4.13 hingga Gambar 4.18, analisis integritas data menunjukkan bahwa relasi *foreign key* antar tabel tidak mengalami korupsi atau pergeseran indeks selama propagasi melalui jaringan *hybrid*. Keberhasilan ini mengonfirmasi bahwa skema replikasi yang diimplementasikan mampu menjamin sifat ACID (*Atomicity, Consistency, Isolation, and Durability*) basis data rumah

sakit. Dengan demikian, *node slave* memiliki salinan data yang sah, mutakhir, dan siap digunakan sebagai landasan proses pencadangan otomatis tingkat lanjut.

4.2.3 Analisis latensi replikasi

Untuk mendapatkan akurasi dalam satuan milidetik (ms), diterapkan instrumen *pt-heartbeat* pada *node Master* dengan interval 0,1 detik (100 ms) untuk menggantikan parameter bawaan MySQL (*Seconds_Behind_Master*) yang terbatas pada satuan detik. Di sisi *node slave*, skrip otomatis mengeksekusi kalkulasi selisih waktu berdasarkan *timestamp* biner murni dan mengekspor hasilnya ke format CSV (*Comma Separated Values*). Selanjutnya data tersebut diolah menggunakan statistik deskriptif berupa rumus *average time* (rata-rata) untuk menghitung rata-rata latensi replikasi secara keseluruhan. Pengujian dilakukan melalui dua skenario komprehensif, yaitu kondisi normal (*idle*) dan kondisi beban tinggi (saat proses *full backup*).

a. Skenario pengujian kondisi normal (*idle*)

Pada pengujian tahap pertama, dilakukan perekaman latensi selama periode 3 (tiga) menit berturut-turut pada saat antrean transaksi (*traffic*) jaringan *hybrid* berada dalam kondisi lengang atau tidak ada aktivitas input data rekam medis yang masif. Hasil ekstraksi dan agregasi statistik deskriptif dari ratusan baris data *log* tersebut disajikan pada tabel 4.5 berikut.

Tabel 4.5 Hasil Pengukuran Latensi Replikasi pada Kondisi Normal (*Idle*)

Parameter Statistik	Formula Excel	Nilai Hasil Pengujian	Indikasi Sistem
Latensi Minimum (*Min*)	=MIN(B2:B181)	4.3 ms	Kinerja puncak pipa replikasi pada kondisi ideal
Latensi Maksimum (*Max*)	=MAX(B2:B181)	105.3 ms	Jeda tunggu maksimal pemrosesan thread I/O
Rata-rata Latensi (*Average*)	=AVERAGE(B2:B181)	54 ms	Stabilitas rata-rata sinkronisasi data

Melalui perekaman selama ±3 menit, diperoleh total 180 sampel pengujian ($n = 180$) dengan jumlah nilai latensi sebesar 9.728,86 ms. Nilai pemusatan data dihitung menggunakan formula *Average Time* (persamaan 2.1) pada tinjauan teoritis, dengan penjabaran matematis sebagai berikut.

$$Average\ Time = \frac{9.728{,}86}{180} = 54\ ms$$

Dengan pendekatan kalkulasi yang sama terhadap seluruh populasi 180 sampel, didapatkan nilai rata-rata latensi representatif sebesar 54 ms, jauh di bawah batas toleransi 1 detik (1,000 ms). Berdasarkan tabel 4.5, arsitektur replikasi pada kondisi normal menunjukkan performa yang sangat presisi. Fluktuasi yang bergerak antara 4.3 ms hingga 105.3 ms, merupakan karakteristik wajar dari waktu tunggu (*wait time*) proses *Input/Output (I/O)* pada lingkungan VPS.

b. Skenario pengujian kondisi beban tinggi (saat proses *full backup*)

Pengujian tahap kedua dilakukan dengan merekam latensi saat beban kerja tinggi, yaitu melalui eksekusi skrip mariadb-*backup* di latar belakang *node slave*. Pengujian ini krusial untuk membuktikan apakah proses *hot backup* tersebut memicu pelambatan antrean replikasi secara signifikan. Hasil perhitungan statistik dari data perekaman pada skenario beban tinggi disajikan pada tabel 4.6 berikut.

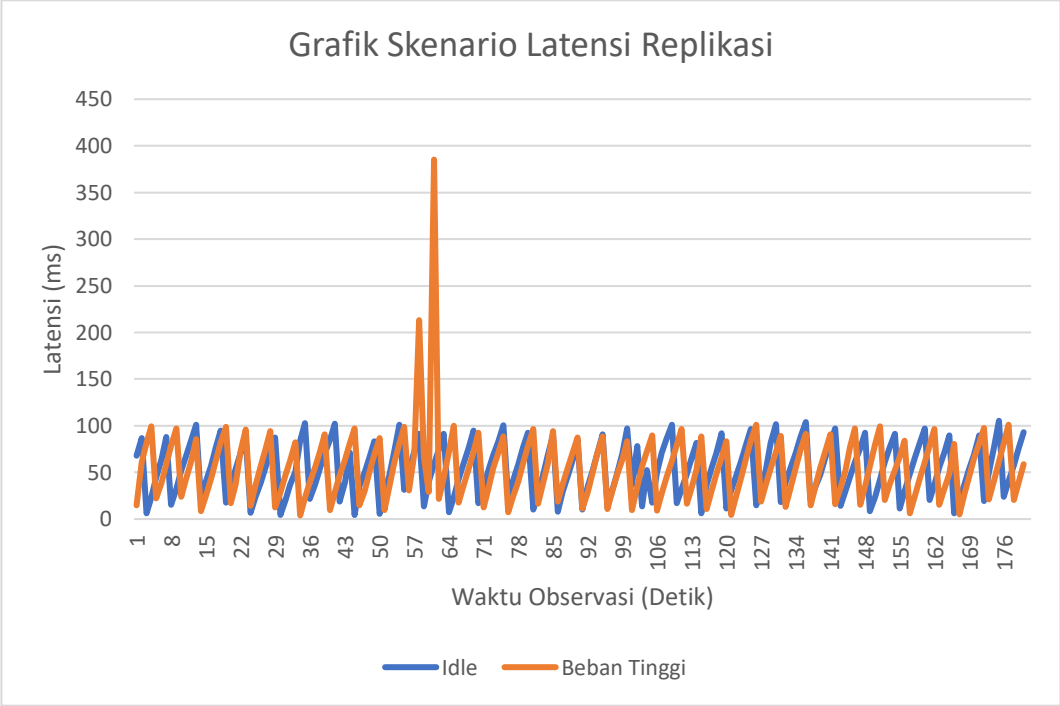
Tabel 4.6 Hasil Pengukuran Latensi Replikasi Saat Proses *Full Backup*

Parameter Statistik	**Formula Excel**	**Nilai Hasil Pengujian**	**Indikasi Sistem**
Latensi Minimum (*Min*)	=MIN(B2:B181)	3.9 ms	Kinerja puncak pipa replikasi pada kondisi ideal
Latensi Maksimum (*Max*)	=MAX(B2:B181)	385.3 ms	Jeda tunggu maksimal pemrosesan *thread I/O*
Rata-rata Latensi (*Average*)	=AVERAGE(B2:B181)	55.5 ms	Stabilitas rata-rata sinkronisasi data

Perekaman selama kurang lebih 3 menit menghasilkan total 180 sampel dan seluruh sampel dijumlahkan dan diperoleh angka 9.991,49 yang nilai pemusatan datanya dihitung menggunakan formula *average time* (persamaan 2.1) pada tinjauan teoritis, dengan penjabaran matematis sebagai berikut.

$$Average\ Time = \frac{9.991{,}49}{180} = 55.5\ ms$$

Meskipun sistem beroperasi di bawah tekanan konkurensi (beban CPU dan *Disk I/O* yang meningkat akibat proses penyalinan data *backup* fisik), parameter rata-rata latensi pada tabel 4.6 menunjukkan angka sebesar 55.5 ms. Terdapat peningkatan latensi maksimum yang mencapai 385.3 ms akibat *resource contention*, namun nilai *Average Time* tetap terjaga pada rentang operasional yang aman.



Gambar 4.19 Grafik Komparasi Fluktuasi Latensi

Data gambar 4.19 yang bersumber dari pencacatan instrumen *pt-heartbeat* dengan format CSV selama 3 menit observasi, menunjukkan kenaikan rata-rata latensi yang sangat minim dari kondisi normal (54 ms) ke skenario beban tinggi saat *full backup* (55.5 ms). Lonjakan maksimum sesaat hingga 385.3 ms mengindikasikan adanya kontensi sumber daya *Disk I/O* di *node slave*, yang

memicu antrean singkat pada *SQL Thread*. Namun, grafik yang segera kembali stabil membuktikan keandalan *catch-up mechanism* dalam mengisolasi beban pencadangan di server cadangan, sehingga operasional input data pada *node master* tetap berjalan *real-time* tanpa risiko penguncian tabel (*table locking*).

4.2.4 Analisis efisiensi durasi otomatisasi *backup*

Analisis durasi otomatisasi *backup* bertujuan mengevaluasi kinerja operasional mariadb-*backup* dan *cron job* pada Linux Ubuntu Server. Dalam ekosistem SIMRS, jendela eksekusi *backup* (*backup execution window*) merupakan metrik vital karena durasi yang terlalu lama dapat memicu degradasi performa sistem akibat lonjakan beban kerja *Disk Read/Write I/O* serta konsumsi memori server secara berkelanjutan.

Sesuai dengan konfigurasi *crontab* Linux yang telah direalisasikan pada tabel 4.3, memisahkan analisis durasi ini menjadi 2 (dua) kategori pencadangan, yaitu *full backup* harian yang dieksekusi setiap pukul 02:00 WIB dan *incremental backup* berkala yang dieksekusi pada pukul 09:00, 15:00, dan 21:00 WIB.

a. Analisis skenario *full backup* (pukul 02:00 WIB)

Proses *full backup* melibatkan penyalinan seluruh fisik berkas data basis data secara utuh. Pengujian dilakukan dengan mengamati catatan waktu (*timestamp*) mulai dan selesai eksekusi pada berkas *log* mariadb-*backup* selama beberapa sesi pengujian. Hasil pencatatan durasi waktu untuk skenario *full backup* dirangkum dalam tabel 4.7 berikut.

Tabel 4.7 Hasil Pengukuran Durasi Otomatisasi *Full Backup*

Sesi Pengujian (Backup ke-*i*)	**Waktu Mulai (WIB)**	**Waktu Selesai (WIB)**	**Durasi Penyelesaian (Detik)**	**Status Sistem**
16 Juni 2026	02:00:00	02:00:10	10	Sukses
17 Juni 2026	02:00:01	02:00:09	8	Sukses
18 Juni 2026	02:00:01	02:00:09	8	Sukses
19 Juni 2026	02:00:00	02:00:09	9	Sukses
20 Juni 2026	02:00:00	02:00:10	10	Sukses

Lanjutan Tabel 4.7

21 Juni 2026	02:00:01	02:00:10	9	Sukses
22 Juni 2026	02:00:00	02:00:10	10	Sukses

Berdasarkan penerapan rumus *average time* terhadap data populasi *full backup* (n = 7) pada tabel 4.7, maka perhitungan kuantitatifnya dihitung menggunakan formula *average time* (persamaan 2.1) pada tinjauan teoritis, dengan penjabaran matematis sebagai berikut.

$$Average\ Time = \frac{10 + 8 + 8 + 9 + 10 + 9 + 10}{7}$$

$$Average\ Time = \frac{64}{7} = 9{,}1\ detik$$

Dari hasil perhitungan tersebut, diperoleh rata-rata durasi penyelesaian untuk satu siklus *full backup* data rekam medis pasien adalah selama 9,1 detik (atau 9100 ms). Kecepatan eksekusi ini dinilai sangat efisien dan aman bagi lingkungan produksi rumah sakit karena waktu operasional server yang singkat meminimalkan beban kerja di pusat data.

b. Analisis skenario *incremental backup* (pukul 09:00, 15:00, dan 21:00 WIB)

Berbeda dengan *full backup*, metode *incremental backup* hanya menyalin berkas delta atau perubahan data klinis yang terjadi sejak pencadangan terakhir dilakukan. Pengujian ini krusial untuk membuktikan efisiensi ruang penyimpanan dan kecepatan waktu proses di tengah jam sibuk pelayanan rumah sakit. Hasil pencatatan durasinya dipaparkan pada tabel 4.8 berikut.

Tabel 4.8 Hasil Pengukuran Durasi Otomatisasi *Incremental Backup*

Sesi Pengujian (Backup ke-i)	**Jadwal Eksekusi**	**Waktu Mulai (WIB)**	**Waktu Selesai (WIB)**	**Durasi Penyelesaian (Detik)**
16 Juni 2026	Siklus Pagi	09:00:01	09:00:10	9
	Siklus Sore	15:00:01	15:00:09	8
	Siklus Malam	21:00:01	21:00:09	8

Lanjutan Tabel 4.8

17 Juni 2026	Siklus Pagi	09:00:00	09:00:09	9
	Siklus Sore	15:00:01	15:00:09	8
	Siklus Malam	21:00:00	21:00:08	8
18 Juni 2026	Siklus Pagi	09:00:01	09:00:14	13
	Siklus Sore	15:00:01	15:00:09	8
	Siklus Malam	21:00:01	21:00:09	8

Melalui kalkulasi statistik menggunakan formula *average time* (persamaan 2.1) pada tinjauan teoritis terhadap sampel *incremental backup* (n = 9), diperoleh perhitungan sebagai berikut.

$$Average\ Time = \frac{9 + 8 + 8 + 9 + 8 + 8 + 13 + 8 + 8}{9}$$

$$Average\ Time = \frac{79}{9} = 8{,}7\ detik$$

Berdasarkan komparasi antara durasi eksekusi *full backup* rata-rata (9,1 detik) dengan *incremental backup* rata-rata (8,7 detik), tingkat efisiensi waktu pemrosesan yang dicapai sistem dapat diukur. Tingkat efisiensi waktu (*time efficiency rate*) ini dihitung menggunakan formula persentase perbandingan selisih waktu operasional terhadap waktu dasar (persamaan 2.2) pada tinjauan teoritis dan diperoleh perhitungan sebagai berikut.

$$Tingkat\ Efisiensi = \frac{T_{full} - T_{incremental}}{T_{full}}\ \text{X}\ 100\%$$

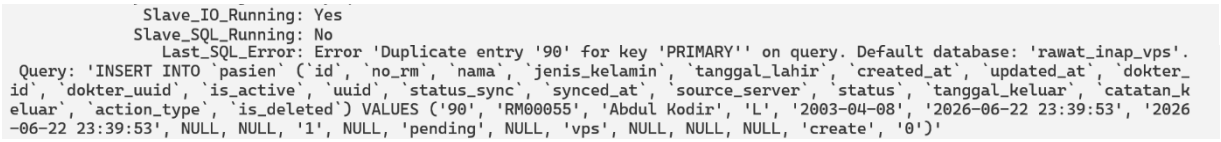
$$Tingkat\ Efisiensi = \frac{9{,}1 - 8{,}7}{9{,}1}\ \text{X}\ 100\% = 4{,}39\%$$

Hasil perhitungan kuantitatif di atas menunjukkan bahwa penerapan algoritma *incremental backup* pada jam sibuk pelayanan rumah sakit (pukul 09:00, 15:00, dan 21:00 WIB) berhasil memberikan tingkat efisiensi waktu eksekusi sebesar 4,39% per siklus pencadangan jika dibandingkan dengan eksekusi penyalinan fisik secara penuh. Meskipun persentase pemangkasan waktu ini berada pada skala hitungan detik, dalam ekosistem produksi rumah sakit dengan laju

pertumbuhan akumulasi data medis yang masif dan eksponensial di masa depan, rasio efisiensi ini akan sangat esensial dalam mereduksi utilisasi beban baca-tulis perangkat keras (*Disk I/O*) peladen sekunder.

4.2.5 Pengujian penanganan kegagalan replikasi

Simulasi anomali kritis berupa berhentinya sinkronisasi akibat konflik integritas data dilakukan dengan mengondisikan pelanggaran hak akses *read-only* pada *node slave* secara sengaja, dengan melakukan injeksi data pemeriksaan pasien ilegal (misalnya ID: 90, No. RM: RM00055) secara langsung ke dalam basis data *node slave*. Setelah itu, simulasi dilanjutkan dengan menginput data pasien sah menggunakan *Primary Key* yang sama (ID: 90) melalui aplikasi SIMRS terminal rawat inap yang terhubung ke *node master*. Injeksi paralel ini secara logis memicu benturan identitas (*duplicate key*) ketika *relay log* pada *slave* mencoba menerapkan transaksi dari *master*, yang menyebabkan proses replikasi terhenti meski kedua server berstatus *running*. Hasil pemantauan parameter saat konflik terjadi didokumentasikan pada gambar 4.20 dan tabel 4.9 berikut.



```
             Slave_IO_Running: Yes
            Slave_SQL_Running: No
               Last_SQL_Error: Error 'Duplicate entry '90' for key 'PRIMARY'' on query. Default database: 'rawat_inap_vps'.
 Query: 'INSERT INTO `pasien` (`id`, `no_rm`, `nama`, `jenis_kelamin`, `tanggal_lahir`, `created_at`, `updated_at`, `dokter_
id`, `dokter_uuid`, `is_active`, `uuid`, `status_sync`, `synced_at`, `source_server`, `status`, `tanggal_keluar`, `catatan_k
eluar`, `action_type`, `is_deleted`) VALUES ('90', 'RM00055', 'Abdul Kodir', 'L', '2003-04-08', '2026-06-22 23:39:53', '2026
-06-22 23:39:53', NULL, NULL, '1', NULL, 'pending', NULL, 'vps', NULL, NULL, NULL, 'create', '0')'
```

Gambar 4.20 *Log* Eror *Duplicate Entry* pada Status Replikasi *Node Slave*

Berdasarkan *log* eror *Duplicate Entry* yang terekam pada gambar 4.20 di atas, observasi lebih lanjut dilakukan untuk menganalisis dampak anomali tersebut terhadap status *thread* replikasi, yang tahapan evaluasinya diuraikan secara rinci pada tabel 4.9 berikut.

Tabel 4.9 Evaluasi Skenario Kegagalan Sinkronisasi (Konflik Data)

Tahap Observasi	**Tindakan / Kondisi Sistem**	**Status *Slave_IO_Running***	**Status *Slave_SQL_Running***	**Indikasi Sistem & Output Observasi**
1	Injeksi data ilegal ke *node slave*	**Yes**	**Yes**	Replikasi masih berjalan normal, menunggu *log* baru dari *Master*.

Lanjutan Tabel 4.9

2	Injeksi data sah ke *node master* dengan ID yang sama	**Yes**	**No**	*Thread* SQL terhenti karena benturan data. Memunculkan peringatan: **Error 1062: Duplicate entry '90' for key 'PRIMARY'.**
3	Pemulihan (*Disaster Recovery*) oleh *Administrator*	**Yes**	**Yes**	Data ilegal di *Slave* dihapus secara manual. Sistem melakukan *catch-up* sinkronisasi untuk data yang sempat tertahan.

Hasil observasi tabel 4.9 menunjukkan bahwa saat konflik *Primary Key* terjadi, parameter *Slave_IO_Running* tetap berstatus *Yes*, tetapi *Slave_SQL_Running* berubah menjadi *No* akibat *error 1062*. Temuan ini mengungkap celah arsitektural krusial, yakni kegagalan replikasi yang tidak ditangani akan membuat rutinitas *cron job* menghasilkan berkas cadangan berisi data usang (*stale data*) karena *log* terbaru tertahan di *relay log*. Untuk memulihkan sistem tanpa kehilangan antrean data pasien, dilakukan intervensi server cadangan melalui prosedur *disaster recovery* dengan menghentikan replikasi sementara (**STOP SLAVE;**), menghapus baris data penyebab konflik (**DELETE FROM pasien WHERE id = 90;**), dan mengaktifkan kembali mesin sinkronisasi (**START SLAVE;**).

4.2.6 Pengujian ketersediaan (*failover*)

Pengujian ketersediaan (*failover*) dilakukan secara eksperimental untuk memverifikasi ketangguhan sistem dalam menjaga layanan rekam medis saat terjadi gangguan pada server utama. Simulasi kegagalan terkontrol dilakukan dengan mematikan layanan MySQL pada *node master* secara paksa menggunakan perintah **systemctl stop mariadb.** Segera setelah server utama padam, perilaku komponen replikasi dan fungsionalitas akses data pada aplikasi diuji untuk memastikan *node*

slave siap mengambil alih peran melayani kueri baca (*read-only*), yang hasilnya dirangkum pada tabel 4.10 berikut.

Tabel 4.10 Hasil Pengujian Skenario Ketersediaan (*Failover*) Sistem

Tahap	**Kondisi *Node Master***	**Aktivitas Akses Aplikasi**	**Parameter Status Replikasi *Node Slave***	**Hasil Observasi (*Output*)**	**Status**
1	*Active* (*Running*)	Kueri Tulis (*Write*) & Baca (*Read*)	*Slave_IO_Running: Yes* *Slave_SQL_Running: Yes*	Data tersimpan pada *master* dan tersinkron-isasi sempurna ke *slave*.	Sukses
2	*Down* (*Stopped*)	Kueri Tulis (*Write*) Baru	*Slave_IO_Running: Connecting* *Slave_SQL_Running: Yes*	Kueri gagal dieksekusi; koneksi ke server *master* terputus (*Connection Refused*).	Terkondisi
3	*Down* (*Stopped*)	Kueri Baca (*Read*) Data Lama	*Slave_IO_Running: Connecting* *Slave_SQL_Running: Yes*	Aplikasi berhasil menampil-kan rekam medis pasien mengguna-kan data dari *node slave*.	Sukses

Sebagai bukti validasi dari ringkasan observasi pada tabel 4.10 di atas, kondisi aktual parameter *thread* replikasi pada antarmuka baris perintah (CLI) saatpeladen utama (*node master*) berstatus aktif disajikan pada gambar 4.21 dan gambar 4.22 berikut.

```
MariaDB [(none)]> SHOW SLAVE STATUS\G;
*************************** 1. row ***************************
               Slave_IO_State: Waiting for master to send event
                  Master_Host: [redacted]
                  Master_User: [redacted]
                  Master_Port: 3306
                Connect_Retry: 60
              Master_Log_File: mysql-bin.000007
          Read_Master_Log_Pos: 342
               Relay_Log_File: mysqld-relay-bin.000041
                Relay_Log_Pos: 641
        Relay_Master_Log_File: mysql-bin.000007
             Slave_IO_Running: Yes
            Slave_SQL_Running: Yes
              Replicate_Do_DB:
```

Gambar 4.21 Status *Slave* Ketika *Node Master* Aktif

Sebaliknya, ketika layanan basis data pada peladen utama (*node master*) dimatikan secara paksa untuk menyimulasikan skenario kegagalan, perubahan respons dan status *thread* replikasi pada *node slave* dapat diamati secara langsung pada gambar 4.22 berikut.

```
MariaDB [(none)]> SHOW SLAVE STATUS\G;
*************************** 1. row ***************************
               Slave_IO_State: Reconnecting after a failed master event read
                  Master_Host: [redacted]
                  Master_User: [redacted]
                  Master_Port: 3306
                Connect_Retry: 60
              Master_Log_File: mysql-bin.000006
          Read_Master_Log_Pos: 312412
               Relay_Log_File: mysqld-relay-bin.000039
                Relay_Log_Pos: 312711
        Relay_Master_Log_File: mysql-bin.000006
             Slave_IO_Running: Connecting
            Slave_SQL_Running: Yes
              Replicate_Do_DB:
```

Gambar 4.22 Status *Slave* Ketika *Node Master Down*

Berdasarkan data observasi pada gambar 4.21, gambar 4.22, dan tabel 4.10, saat *node master* lumpuh, transaksi penulisan data baru dihentikan sementara, namun pemanggilan kembali rekam medis historis (*read-only retrieval*) tetap berjalan normal karena sistem otomatis mengalihkan *pointer* koneksi baca ke alamat IP *node slave*. Pengalihan beban ini berhasil menjamin operasional diagnosis tenaga medis menggunakan replika data terakhir yang sinkron. Keberhasilan pengujian *failover* ini membuktikan secara pengamatan bahwa topologi basis data terdistribusi pada jaringan *hybrid* mampu meminimalkan risiko *single point of failure* sekaligus bisa menjadi solusi *disaster recovery* yang andal.

4.3 Pembahasan

Bagian pembahasan ini menjelaskan hasil pengujian dan analisis performa sistem replikasi basis data dan otomatisasi pencadangan yang telah diimplementasikan pada jaringan *hybrid*. Evaluasi difokuskan pada tingkat konsistensi data, stabilitas latensi sinkronisasi, efisiensi waktu eksekusi *backup*, serta resiliensi sistem dalam skenario kegagalan (*failover*).

4.3.1 Analisis keandalan dan konsistensi replikasi *master-slave*

Penerapan topologi *master-slave* pada jaringan *hybrid* rumah sakit berhasil mendistribusikan data dengan tingkat yang konsisten dan nilai selisih (*delta*) nol berdasarkan uji injeksi 50 *records* rekam medis. Keberhasilan transmisi tanpa korupsi *foreign key* atau pergeseran indeks ini membuktikan bahwa arsitektur sistem mampu menjamin sifat ACID (*Atomicity, Consistency, Isolation, and Durability*). Kinerja tersebut menegaskan bahwa penggunaan MariaDB pada lingkungan VPS efektif menghilangkan risiko *single point of failure*, sekaligus memenuhi tuntutan ketersediaan data berkelanjutan dalam modernisasi SIMRS.

4.3.2 Latensi jaringan terhadap proses *hot hackup*

Pengukuran latensi menggunakan instrumen *pt-heartbeat* membuktikan kualitas dan resiliensi jaringan *hybrid* dalam menangani lalu lintas data medis. Pada kondisi normal (*idle*), rata-rata latensi tercatat sebesar 54 ms dengan nilai maksimum 105.3 ms. Saat skenario beban tinggi akibat eksekusi *full backup* di *node slave*, terjadi kontensi sumber daya *Disk I/O* dan CPU yang memicu lonjakan latensi maksimum secara tajam hingga 385.3 ms. Namun, analisis menggunakan *average time* menunjukkan bahwa akumulasi rata-rata latensi hanya bergeser sedikit ke angka 55.5 ms (selisih kenaikan hanya 1.5 ms).

Stabilitas rata-rata latensi yang berada jauh di bawah ambang batas kritis (1,000 ms) ini mengonfirmasi dua keberhasilan teknis secara bersamaan, efektivitas *catch-up mechanism* yang mempercepat pemulihan antrean replikasi tanpa memicu kegagalan pipa komunikasi, serta keberhasilan operasi *non-locking* dari mariadb-*backup* yang mencegah efek penguncian tabel (*table locking*). Hasil ini membuktikan bahwa infrastruktur *master-slave* Penulis mampu menjamin

operasional input data klinis secara *real-time* tanpa disrupsi (*zero-downtime*), terlepas dari adanya aktivitas pencadangan berat di latar belakang.

4.3.3 Efisiensi otomatisasi *backup* menggunakan *average time*

Otomatisasi menggunakan utilitas *cron job* berhasil menggantikan keharusan intervensi manual administrator dalam mengamankan data medis. Pengukuran melalui metode *average time* membuktikan rata-rata durasi *full backup* harian sebesar 9,1 detik dan *incremental backup* berkala sebesar 8,7 detik. Berdasarkan komparasi tersebut, sistem arsitektur yang dibangun terbukti menghasilkan tingkat efisiensi waktu komputasi sebesar 4,39% pada jam-jam operasional puncak layanan rawat inap. Durasi eksekusi yang konsisten berada di bawah ambang batas kritis 10 detik ini merepresentasikan rasio efisiensi yang tinggi, sehingga secara efektif mampu mencegah degradasi performa (*bottleneck*) akibat utilisasi *Disk Read/Write I/O* yang berlebihan pada peladen cadangan.

4.3.4 Implikasi kegagalan sinkronisasi replikasi

Pengujian *fault injection* mengungkap celah krusial berupa anomali redudansi data kedaluwarsa, konflik integritas data (*error 1062*) menyebabkan proses aplikatif kueri (*Slave_SQL_Running*) lumpuh total meskipun pipa komunikasi data tetap aktif. Kondisi ini menahan data rekam medis baru di dalam *relay log* tanpa diterapkan ke tabel produksi cadangan. Karena utilitas *cron job* bekerja murni berdasarkan penjadwalan waktu tanpa pemantauan kesehatan internal basis data, skrip mariadb-*backup* akan tetap dieksekusi otomatis pada pukul 02:00 WIB dan menghasilkan status *Success*. Hal ini menciptakan risiko tinggi berupa ilusi keamanan (*false sense of security*), sebab berkas cadangan yang terbentuk sebenarnya berisi data usang (*stale data*) yang telah mengalami desinkronisasi dari server utama.

Temuan ini membuktikan bahwa otomatisasi *backup* memerlukan mekanisme pengawasan status replikasi (*replication state monitoring*) dan sistem peringatan dini *real-time* guna menjamin validitas serta kemutakhiran data rekam medis pasien saat terjadi bencana (*disaster recovery*).

4.3.5 Ketersediaan data tinggi (*high availability*) dan *failover*

Uji simulasi kegagalan (*failover test*) menjadi alasan akhir keandalan arsitektur ini bisa sebagai solusi pemulihan bencana (*disaster recovery*). Saat layanan basis data utama lumpuh (*down*), kueri penulisan data baru tertahan demi mencegah inkonsistensi, namun aplikasi rawat inap berhasil dialihkan ke *node slave* untuk mempertahankan fungsionalitas pemanggilan data (*read-only retrieval*). Akses riwayat medis pasien tetap bisa berjalan hampir tanpa adanya jeda (*zero-downtime*), membuktikan bahwa arsitektur jaringan *hybrid* ini berhasil mewujudkan tingkat ketersediaan data tinggi (*high availability*) yang ditargetkan dalam penelitian.

BAB V
PENUTUP

5.1 Simpulan

Berdasarkan serangkaian tahapan penelitian yang telah dilakukan, mulai dari perancangan, pengujian, hingga evaluasi terhadap sistem pemeriksaan pasien rawat inap, ditarik dua simpulan utama yang secara spesifik menjawab rumusan masalah dan mengonfirmasi pencapaian tujuan penelitian sebagai berikut:

a. Implementasi *hot backup* dalam mencegah *table locking*

   Implementasi arsitektur *master-slave* terintegrasi *cron job* dan mariadb-*backup* terbukti efektif menjalankan otomatisasi *hot backup* secara penuh. Sistem berhasil mengisolasi beban kerja pencadangan di peladen cadangan (VPS *slave*), sehingga mencegah efek penguncian tabel (*table locking*) secara mutlak. Ketersediaan layanan (*high availability*) untuk *input* rekam medis di peladen utama (VPS *master*) tetap terjamin tanpa jeda operasional (*zero-downtime*). Namun, sistem ini tetap membutuhkan pengawasan administrator untuk memitigasi risiko pembentukan data usang (*stale data*) apabila terjadi anomali konflik sinkronisasi (*error* 1062).

b. Tingkat efisiensi komputasi dan resiliensi sinkronisasi sistem

   Sistem yang dibangun terbukti mencapai target efisiensi dan stabilitas yang tinggi. Pengukuran metode *average time* mencatat durasi rata-rata *full backup* harian sebesar 9,1 detik dan *incremental backup* sebesar 8,7 detik, yang mengonfirmasi capaian tingkat efisiensi waktu pemrosesan sebesar 4,39%. Sementara itu, pemantauan instrumen *pt-heartbeat* membuktikan resiliensi latensi jaringan yang sangat tangguh, dengan rata-rata waktu tunda (*delay*) 54 ms pada kondisi normal dan hanya bergeser ke 55.5 ms pada kondisi beban tinggi. Angka tersebut memastikan sinkronisasi data medis berjalan presisi dan aman di bawah batas toleransi kritis.

5.2 Saran

Meskipun arsitektur redundansi dan pencadangan data yang dibangun telah memenuhi tujuan penelitian, masih terdapat beberapa batasan yang dapat dikembangkan untuk meningkatkan keandalan ekosistem infrastruktur basis data kesehatan. Oleh karena itu, diajukan beberapa saran strategis sebagai berikut:

a. Implementasi dasbor pemantauan (*monitoring dashboard*)

   Untuk mengurangi ketergantungan pada *Command Line Interface* (CLI), penelitian selanjutnya disarankan mengintegrasikan sistem notifikasi otomatis (seperti Telegram atau Email) dan mengembangkan antarmuka berbasis web (seperti Grafana atau Zabbix). Pengembangan ini bertujuan untuk memvisualisasikan parameter latensi *pt-heartbeat* serta sisa ruang penyimpanan secara interaktif dan *real-time*.

b. Penerapan enkripsi data berstandar medis

   Untuk memperkuat kepatuhan terhadap regulasi privasi pasien, penelitian selanjutnya disarankan menerapkan enkripsi pada berkas cadangan (*encryption at rest*) yang dihasilkan oleh mariadb-*backup*. Selain itu, saluran komunikasi replikasi antar-VPS perlu diperkuat menggunakan sertifikat SSL/TLS guna melindungi integritas transmisi data dari ancaman intersepsi pada jaringan publik.

c. Eksplorasi skenario *multi-region disaster recovery*

   Arsitektur saat ini baru memitigasi *single point of failure* secara lokal pada jaringan *hybrid*. Untuk institusi medis skala besar, penelitian selanjutnya disarankan mengkaji penerapan topologi replikasi asinkron lintas wilayah (*asynchronous multi-region replication*) dengan menempatkan *node* cadangan di pusat data geografis yang berbeda guna mengantisipasi kegagalan infrastruktur skala regional.

DAFTAR PUSTAKA

Andhavarapu, A. (2025). *Building Resilient Distributed Databases: Modern Approaches to High Availability. International Journal of Research in Computer Application and Information Technology (IJRCAIT)*, *8*(1), 1293–1307. https://doi.org/10.34218/ijrcait_08_01_096

D, R., & A, G. (2024). *Survey on Data Replication in Cloud Systems. Web Intelligence*, *22*(1), 83–109. https://doi.org/10.3233/WEB-230087

Devadas, R. S. (2025). *Transforming Healthcare Through AI-Driven Application Modernization and Hybrid Cloud Architecture. Journal of Computer Science and Technology Studies (JCSTS)*, 629–638. https://doi.org/10.32996/jcsts

Dhafa, M., & Kiswanto. (2026). *Evaluasi Backup Cloud Incremental untuk Efisiensi dan Recovery Data Multimedia. Jurnal Ilmiah Teknik Informatika dan Sistem Informasi*, *15*(3). https://doi.org/http://dx.doi.org/10.35889/jutisi.v15i3

Florensia, N. P., Nurulita, M. P., Recita, G., & Parhusip, J. (2024). *Peran Distribusi Nilai Rata-Rata dalam Menganalisis Kualitas Layanan IT dengan Pendekatan Statistika untuk Pengukuran Kinerja Sistem. Jurnal Ilmu Teknik dan Informatika*, *4*, 30–38. https://doi.org/10.51903/teknik

Haryoto, I. S., Firmansyah, G., Tjahjono, B., Widodo, A. M., Akbar, H., & Fatonah, N. S. (2025). *Evaluation and Optimization of MySQL Master-Slave Performance with Quantitative Methods on the Database of Psychology Test Applicants SIM PT XYZ at Polda Metro Jaya. LOCUS JOURNAL: Research & Devotion*, *4*(9), 8896–8905.

Hidayat, H. T., Prihatin, N., Nasir, M., & Astrid, E. (2026). *Design and Analysis of a Mobile Chatbot Application for Hospital Room Availability Using the Bryant Evaluation Method. International Journal of Interactive Mobile Technologies (IJIM)*, *20*(1), 34–51. https://doi.org/10.3991/ijim.v20i01.59507

Hulu, N. K., Tommy, & Handoko, D. (2025). *Optimalisasi Kinerja CMS WordPress Melalui Penerapan Replikasi Database MySQL. Jurnal Multidisiplin dan Sains*, *1*(2), 160–172. https://jurnal.compartdigital.com/index.php/judis

Intania, S., & Rohayani, H. (2025). *Literature Review Study of Data Replication Methods in Distributed System. International Journal of Innovation Research in Education, Technology and Management*, *2*(2), 135–144. https://doi.org/10.61098/ijiretm.v2i2.223

Kag, I. (2023). *A Review Introduction of Database Management System. International Journal of Innovation in Engineering Research & Management*, *10*(2), 42–48.

Kamiswara, A., & Wijaya, A. (2024). *Implementasi Master-Slave Sebagai Koneksi Baca dan Tulis Database Dengan Metode Replication. Jurnal TEKNIKA*, *19*(1), 47–58.

Maulaningrum, P., Mujanah, S., & Fianto, A. Y. A. (2025). *Transformasi Digital di Sektor Kesehatan Tinjauan Literatur tentang Penerapan Teknologi Informasi dalam Manajemen Pelayanan. Jurnal Ilmu Manajemen, Ekonomi dan Kewirausahaan*, *5*(1), 494–503. https://doi.org/10.55606/jimek.v5i1.6399

Mendoza, L. E., Montoya, H., Pérez, M., & Grimán, A. (2003). *Evaluating Critical Success Factors According to Integration Maturity Level*. https://www.researchgate.net/publication/228905392_Evaluating_Critical_Success_Factors_According_to_Integration_Maturity_Level

Muliawan, D., & Iskandar, D. (2023). *Penerapan Replikasi Synchronous pada Aplikasi Data KRS Online pada Universitas Jabal Ghafur Menggunakan PostgreSQL. The Journal of Multidisciplinary Research on Scientific and Advanced*, *1*, 31–45.

Percona LLC. (2025, Mei 19). *Percona XtraBackup 8.0.35-33: Operations manual*. https://learn.percona.com/hubfs/Manuals/Percona_Xtra_Backup/Percona-XtraBackup-8.0/PerconaXtraBackup-8.0.35-33.pdf

Samant, P. S. (2024). *Secure Cloud Services for the Healthcare Industry: Addressing Unique Challenges and Ensuring Compliance. International Journal for Research in Applied Science and Engineering Technology*, *12*(4), 3095–3101. https://doi.org/10.22214/ijraset.2024.60636

Sholihah, I., & Darujati, C. (2022, Desember 13). *Sistem Replikasi Basis Data berdasarkan MySQL menggunakan Container Docker. Majalah Ilmiah Teknologi Elektro*, *21*(2), 209–218. https://doi.org/10.24843/mite.2022.v21i02.p08

Susilo, H., Fauzi, D., Ihksan, M., & Abdillah, N. (2024). *Sosialisasi Strategi Pemanfaatan Replikasi Basis Data (Master-Master dan Master-Slave) untuk Peningkatan Kinerja Aplikasi di RSIA Mutiara Bunda. Jurnal Abdimas Saintika*, *6*(1), 129–136. https://jurnal.syedzasaintika.ac.id

Wibowo, A., Fathimah, A., Waskito, D., & Aprilianto, R. A. (2025). *Cron Job Implementation for Automated Data Processing and Transfer in Cloud Infrastructure. Sainteknol* (Vol. 23, Nomor 1). https://journal.unnes.ac.id/journals/sainteknol

BIODATA PENULIS



Nama	: Al Nazirul Mubin
Tempat, Tanggal Lahir	: Lhokseumawe, 17 Mei 2004
Agama	: Islam
Pekerjaan	: Mahasiswa
Alamat Lengkap	: Batuphat Barat, Kecamatan Muara Satu, Kota Lhokseumawe
No. HP/Email	: 0851 1773 3040 / alnazirul65@gmail.com
Jurusan/Prodi	: Teknologi Informasi dan Komputer / Teknologi Rekayasa Komputer Jaringan
Nama Orang Tua	
a. Ayah	: Saifuddin Hasan
b. Ibu	: Almarhumah Salamah
Pekerjaan Orang Tua	
a. Ayah	: Buruh Harian Lepas
b. Ibu	: Almarhumah
Alamat Orang Tua	: Lueng Daneun, Kecamatan Peusangan Siblah Krueng, Kabupaten Bireuen
Riwayat Pendidikan	
1. SD/Sederajat	: SD Negeri Arun (2016)
2. SMP/Sederajat	: SMP Negeri Arun (2019)
3. SMA/Sederajat	: SMK Negeri 1 Lhokseumawe (2022)
4. Perguran Tinggi	: Politeknik Negeri Lhokseumawe (2026)
Judul Tugas Akhir	: Implementasi dan Analisis Otomatisasi *Backup* Data pada Jaringan *Hybrid* Sistem Pemeriksaan Pasien Rawat Inap di Rumah Sakit